



# Engineering Guide

---

*Architecture, Source Units, Dependencies, Formats, Runtime  
Contracts, Tests, and Release Controls*



Version 1.0

Last changed: August 30, 2026

Windows • Delphi VCL and FireMonkey • Win32 and Win64

## Table of Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| 1. Guide Purpose and Engineering Vocabulary                       | <b>1</b>  |
| 2. Product Scope, Non-Negotiable Invariants, and Trust Boundaries | <b>3</b>  |
| 3. Repository, Build, and Distribution Layout                     | <b>4</b>  |
| 4. End-to-End Architecture and Ownership                          | <b>5</b>  |
| 5. Core Domain Model and Catalog Lifecycle                        | <b>7</b>  |
| 6. Workspace, Persistence, and Recovery                           | <b>8</b>  |
| 7. Project Detection and Canonical Scanning                       | <b>9</b>  |
| 8. Context, Terminology, Translation Memory, and Glossary         | <b>9</b>  |
| 9. Provider Pipeline and Credential Security                      | <b>10</b> |
| 10. Localization Review, Measurement, Layout, and Hyphenation     | <b>11</b> |
| 11. Validation and Runtime Pack Export                            | <b>12</b> |
| 12. Offline Runtime Manager and Pack Admission                    | <b>12</b> |
| 13. VCL Runtime Application and Lifecycle                         | <b>13</b> |
| 14. FMX Runtime Application and Lifecycle                         | <b>13</b> |
| 15. Components, Design Packages, and Designer Contract            | <b>14</b> |
| 16. ComponentSource and Target dependencies                       | <b>15</b> |
| 17. Integration, Build, Deployment, and Project Boundaries        | <b>16</b> |
| 18. Studio Forms, State Machines, and Self-Localization           | <b>16</b> |
| 19. Directionality, HTML, Dynamic Text, and Reversible Layout     | <b>17</b> |
| 20. Diagnostics, Failure Containment, and Performance             | <b>18</b> |
| 21. Security and Privacy Engineering                              | <b>18</b> |
| 22. Test Architecture and Release Gates                           | <b>19</b> |
| 23. Production Pascal Unit Reference                              | <b>21</b> |
| 24. Test, Spike, and Contract Pascal Program Reference            | <b>59</b> |
| 25. Delphi Project and Package File Reference                     | <b>77</b> |
| 26. JSON Schema and Form Resource Reference                       | <b>79</b> |

|                                                           |           |
|-----------------------------------------------------------|-----------|
| 27. Build, Verification, and Documentation Tool Reference | <b>83</b> |
| 28. Dependency Graph Reading Guide                        | <b>86</b> |
| 29. Safe Change Procedure                                 | <b>87</b> |
| 30. Engineering Release Checklist                         | <b>88</b> |
| 31. Quick Path and Identifier Reference                   | <b>89</b> |

## 1. Guide Purpose and Engineering Vocabulary

This guide explains the complete engineering system before presenting the file-by-file reference. It is written for Delphi developers maintaining the Studio, the VCL/FMX runtime components, provider integration, scanners, review tooling, packages, deployment pipeline, tests, and documentation.

A term is introduced here before later chapters depend on it. The final reference chapters describe every production PAS unit individually, every test/spike/contract Pascal program, every Delphi project/package file, each JSON schema and FMX/DFM resource, and the important build and verification tools.

| Term                         | Definition                                                                                                                                                      | Ownership consequence                                                                            |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>ApplicationId</b>         | Stable exact identity shared by selected Delphi project, workspace, catalog, pack, runtime manager, and preference.                                             | Never derive it from a translated caption or arbitrary folder alias.                             |
| <b>Development catalog</b>   | Full editable translation model containing stable keys, source/target text, status, origin, context, locale facts, checksums, scan provenance, and review data. | Studio-owned; not deployed directly as the compact runtime contract.                             |
| <b>Canonical source pack</b> | Validated runtime pack containing the authored source language.                                                                                                 | Required for switch-back, compatibility comparison, and safe fallback.                           |
| <b>Runtime pack</b>          | Compact validated JSON used by the offline target application.                                                                                                  | Must match ApplicationId, framework, schema/version, locale, checksum, and source-pack contract. |
| <b>Stable key</b>            | Deterministic identity such as <code>form.component.property</code> or <code>unit.resourcestring</code> .                                                       | Preserves translation across scans when ownership and source remain compatible.                  |

| Term                       | Definition                                                                                                                        | Ownership consequence                                                                      |
|----------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Semantic contract</b>   | Explicit application-owned key for dynamic text, templates, HTML, or other content not represented by one designer property.      | Allows safe scanning/application without translating arbitrary live data.                  |
| <b>Manager</b>             | One nonvisual VCL or FMX component that owns runtime discovery, selected locale, lifecycle, events, errors, and pack application. | Exactly one primary manager normally supervises the application.                           |
| <b>Selector</b>            | Designer-visible VCL/FMX combo box connected to a manager.                                                                        | Lists only admitted packs and delegates language switching to the manager.                 |
| <b>Applicator</b>          | Framework-specific walker that applies text, layout, direction, and restoration to a VCL or FMX object tree.                      | Contains framework behavior; provider and catalog code remain framework-neutral.           |
| <b>Designer baseline</b>   | Snapshot of source-language text, geometry, alignment, direction, and relevant control state.                                     | Restored before every language transition so changes are reversible and do not accumulate. |
| <b>Localization Review</b> | Operator window and artifact package for linguistic findings, glossary, suggestions, layout proposals, and decisions.             | Human decisions remain separate from structural validation.                                |
| <b>ComponentSource</b>     | Complete current subset of runtime/component PAS units generated for one target framework.                                        | Must be copied as one compatible set; partial copying creates interface drift.             |
| <b>Workspace</b>           | Per-ApplicationId Studio state under %LOCALAPPDATA%\DelphiAppTranslationStudio\Workspaces.                                        | Keeps development data outside target source and executable folders.                       |

| Term           | Definition                                                        | Ownership consequence                                                                          |
|----------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Pack admission | Runtime validation performed before a pack appears as selectable. | Invalid, wrong-app, wrong-framework, wrong-version, or incompatible packs never become active. |

## 2. Product Scope, Non-Negotiable Invariants, and Trust Boundaries

- Target Pascal, DFM, FMX, DPR, and DPROJ remain read-only in the recommended Wizard/component workflow.
- Forms and published component properties remain designer-authored and editable in Object Inspector; runtime-only UI construction is not the normal integration mechanism.
- The development Studio may use DeepL or Google online; the deployed VCL/FMX application is offline and contains no provider credential.
- A language transition is reversible: restore the designer/source baseline first, then apply the selected pack once. Never translate the previous translation.
- Live application data, identifiers, object names, file paths, API names, numeric values, selection/focus, and connection state are not language-pack text unless an explicit semantic contract says otherwise.
- VCL and FMX share catalogs, packs, provider, validation, and manager core but use separate applicators, lifecycle hooks, direction rules, text measurement, and design packages.
- Stable keys and source text preserve unchanged work. Provider calls are limited to eligible unresolved entries; reviewed and approved work is never silently overwritten.
- All persistence that can affect catalogs, packs, preferences, settings, glossaries, deployment, or layout decisions is atomic and recoverable.
- RTL changes paragraph/control direction and alignment while preserving protected LTR tokens and explicit application-owned layout.
- No project-specific V5/V6 or named-application exception belongs in universal scanner, runtime, or layout code.

**Trust boundary.** The Studio trusts only explicitly selected project source and validated provider responses; the runtime trusts only admitted local packs. Neither side treats arbitrary JSON, user data, provider text, or an unverified executable folder as safe input.

### 3. Repository, Build, and Distribution Layout

| Folder                    | Engineering ownership                                                                                          | What must not be mixed into it                            |
|---------------------------|----------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| <b>source\core</b>        | framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction  | FMX/VCL controls, HTTP UI, or target-specific exceptions. |
| <b>source\scan</b>        | project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging | Runtime mutation or provider secrets.                     |
| <b>source\provider</b>    | DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation      | Target runtime or design-time registration.               |
| <b>source\review</b>      | localization findings, layout proposals, text measurement, and application-owned string review                 | Silent application of unapproved geometry.                |
| <b>source\validation</b>  | structural catalog checks that protect runtime export                                                          | Linguistic approval decisions.                            |
| <b>source\runtime</b>     | offline pack loading, preference, translation application, layout restoration, templates, and splash handling  | Provider networking or Studio forms.                      |
| <b>source\components</b>  | designer-visible language-manager components, lifecycle adapters, and language selectors                       | Studio workflow or catalog editing.                       |
| <b>source\design</b>      | RAD Studio Tool Palette registration and design-time package exposure                                          | Code required by deployed applications.                   |
| <b>source\integration</b> | component-kit generation, controlled Delphi project integration, build, and deployment                         | Unbounded source rewriting.                               |

| Folder                        | Engineering ownership                                                                                    | What must not be mixed into it                          |
|-------------------------------|----------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>source\studio</b>          | the FMX operator interface, Setup Wizard state machine, Localization Review, and Studio self-translation | Core algorithms that tests/runtime need without FMX UI. |
| <b>packages</b>               | Runtime and design package projects.                                                                     | Application-specific compiled BPL copies.               |
| <b>bin / dcu</b>              | Platform/configuration output.                                                                           | Authoritative source or hand-edited files.              |
| <b>docs\guides / docs\pdf</b> | Editable DOCX guides and print PDFs.                                                                     | Runtime dependencies or credentials.                    |
| <b>export</b>                 | Generated review, pack, kit, and source-distribution artifacts.                                          | Developer secrets or canonical source history.          |
| <b>tools / tools\tests</b>    | Build, verification, docs, contracts, test executables, and fixtures.                                    | Production code reachable only from tests.              |

DelphiAppTranslationStudio.dpr/dproj are the FMX Studio entry point and project metadata. Output is partitioned by platform/configuration. Package projects separate framework-neutral runtime, VCL runtime, FMX runtime, and IDE-only VCL/FMX registration.

Source distributions must contain complete source/package/schema/tool/document sets and must exclude secrets, Local AppData workspaces, provider keys, transient test workspaces, DCUs, EXEs, and unreviewed proprietary catalogs unless explicitly intended.

## 4. End-to-End Architecture and Ownership

| Stage         | Input              | Owner                     | Output / next boundary                                           |
|---------------|--------------------|---------------------------|------------------------------------------------------------------|
| <b>Detect</b> | Selected DPR/DPROJ | DAT.Core.ProjectDetection | Framework, ApplicationId, targets, forms, units, output metadata |



| Stage            | Input                                                  | Owner                                        | Output / next boundary                                                  |
|------------------|--------------------------------------------------------|----------------------------------------------|-------------------------------------------------------------------------|
| <b>Scan</b>      | Saved DFM/FMX/PAS and authorized resources             | DAT.Scan.*                                   | TProjectScanResult with items, diagnostics, snapshots, and counts       |
| <b>Merge</b>     | Scan result plus prior catalog                         | DAT.Scan.CatalogMerge                        | Preserved/new/changed/obsolete development catalog                      |
| <b>Resolve</b>   | Entries, domain profile, glossary, memory, terminology | DAT.Core.* and DAT.Provider.*                | Eligible provider inputs and deterministic repairs                      |
| <b>Translate</b> | Bounded text/context batches and credential            | DAT.Provider.Client / DAT.Core.AITranslation | Machine drafts with status/origin and diagnostics                       |
| <b>Review</b>    | Translated catalog and source geometry                 | DAT.Review.* / LocalizationReview form       | Findings, glossary, suggestions, layout decisions, HTML/JSON package    |
| <b>Validate</b>  | Complete development catalog                           | DAT.Validation.Catalog                       | Blocking errors and review warnings                                     |
| <b>Export</b>    | Validated catalog                                      | DAT.Core.RuntimePack                         | Canonical source pack and translated runtime pack                       |
| <b>Integrate</b> | Project profile and packs                              | DAT.Integration.ComponentPackage             | ComponentSource, manifest, README, deployment script, completion report |
| <b>Admit</b>     | Pack files beside target EXE                           | DAT.Runtime.LanguagePack / Manager           | Selectable compatible language descriptors                              |

| Stage                                                                                                                                                                                                               | Input                              | Owner                               | Output / next boundary                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|-------------------------------------|---------------------------------------------------------|
| Apply                                                                                                                                                                                                               | Selected pack and open/later forms | DAT.Runtime.VCL/FMX plus components | Immediate reversible translated UI with preserved state |
| <b>Ownership rule.</b> Each stage validates the contract it receives and emits a narrower explicit contract. Later stages must not reach backward into a UI form or provider response to reconstruct missing state. |                                    |                                     |                                                         |

## 5. Core Domain Model and Catalog Lifecycle

DAT.Core.Types is the vocabulary root. TProjectProfile describes target identity/framework/files/targets. TLocaleProfile describes locale facts. TTranslationEntry carries stable identity, source/target text, status, origin, context, ownership, checksums, and review facts. TTranslationCatalog owns project/locale metadata and its entries.

Status is workflow state, not provenance. Origin records how target text arrived; Reviewed/Approved record human decisions. A machine result must not become Approved merely because structural validation passes.

| Catalog event            | Required behavior                                                             | Forbidden behavior                                           |
|--------------------------|-------------------------------------------------------------------------------|--------------------------------------------------------------|
| Same key, same source    | Preserve translation, origin, review, approval, and applicable decisions.     | Retranslate or reset state without explicit operator action. |
| Same key, changed source | Retain prior target for reference and mark source-changed/attention-required. | Treat old target as current approved wording.                |
| New key                  | Add with context, ownership, source snapshot, and unresolved status.          | Borrow another key's approval automatically.                 |
| Missing key              | Retain as obsolete for audit/recovery.                                        | Delete silently.                                             |
| Equivalent duplicate     | Collapse occurrence while retaining provenance/count.                         | Create conflicting duplicate stable keys.                    |
| Semantic recovery        | Recreate known application-owned dynamic contract deterministically.          | Scan arbitrary runtime values as translatable source.        |

## 6. Workspace, Persistence, and Recovery

TTranslationWorkspace roots project state at %LOCALAPPDATA%

\DelphiAppTranslationStudio\Workspaces\<ApplicationId>. Development, Languages, Glossaries, and Deployment separate editable data, deployed-format packs, terminology, and destination settings.

TAtomicTextFile writes a candidate, validates it, preserves a prior known-good version, and replaces the active file atomically. Readers can recover .previous, quarantine corrupt input, and avoid converting a process interruption into an empty or half-written catalog.

| Artifact                   | Writer              | Validation before commit                                        | Recovery behavior                                                                    |
|----------------------------|---------------------|-----------------------------------------------------------------|--------------------------------------------------------------------------------------|
| <b>Development catalog</b> | TCatalogJson        | Schema/domain validation and parse round trip.                  | Previous catalog retained; corrupt input quarantined.                                |
| <b>Runtime pack</b>        | TRuntimePackBuilder | Required metadata, checksums, locale, entries, layout contract. | Invalid pack never replaces the prior admitted copy.                                 |
| <b>Provider settings</b>   | TProviderSettings   | Known provider/plan, bounded timeout/batch, no secret field.    | Defaults and previous valid settings remain available.                               |
| <b>Language preference</b> | TLanguagePreference | Admitted locale code.                                           | Invalid preference falls back to source/system policy.                               |
| <b>Glossary</b>            | TProjectGlossary    | Term/source/target and match-rule validation.                   | Last valid glossary can be restored.                                                 |
| <b>Layout overrides</b>    | TLayoutOverrides    | Application/locale/form/control identity and numeric bounds.    | Invalid overrides are ignored/quarantined rather than moving controls unpredictably. |

## 7. Project Detection and Canonical Scanning

TProjectDetector establishes the source closure before scanning. TProjectScanner orchestrates form and Pascal scanners, rules, context, domain profile, quality analysis, progress/cancellation, source snapshots, and the final result.

TTextFormScanner reads text DFM/FMX structure and eligible string properties.

TPascalResourceStringScanner reads resourcestring declarations and approved runtime assignments.

TScanRuleSet is the universal policy boundary; it must not contain named-project or version-specific exceptions.

| Count                                                                                                                                                                                                     | Meaning                                                           | Why it is separate                                                       |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Raw observations</b>                                                                                                                                                                                   | Direct form/Pascal/resource observations.                         | Shows scanner input volume before merge behavior.                        |
| <b>Recovered semantic contracts</b>                                                                                                                                                                       | Explicit dynamic/application-owned strings added by stable rules. | Dynamic coverage is auditable rather than hidden in code.                |
| <b>Equivalent duplicates collapsed</b>                                                                                                                                                                    | Repeated equivalent ownership represented once.                   | Prevents inflated translation counts without losing occurrence evidence. |
| <b>Unique catalog entries</b>                                                                                                                                                                             | Final stable-key records after recovery/collapse.                 | This is the canonical Wizard/Maintenance headline.                       |
| <b>Scan snapshot invariant.</b> Final processing must reject a source/form save that occurred after the accepted scan. Exporting a stale snapshot can omit new text even when every later stage succeeds. |                                                                   |                                                                          |

## 8. Context, Terminology, Translation Memory, and Glossary

Context and terminology are layered because provider text alone is insufficient for short interface strings. TScanContextAnalyzer records UI role, semantic concept, description, confidence, and surrounding evidence. TDomainProfiler builds project vocabulary and senses. TProjectGlossary and TSharedDictionary carry approved local and shared terms. TTranslationMemory reuses reviewed prior segments with provenance.

Resolution order must preserve the most specific trusted evidence. Project glossary outranks shared/general terminology; reviewed contextual memory outranks an unreviewed machine draft; protected tokens and placeholders are never rewritten as terminology.

| Layer                                   | Strength                                                  | Allowed effect                                                                            |
|-----------------------------------------|-----------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>Protected token/placeholder</b>      | Absolute structural constraint                            | Preserve exactly and validate restoration.                                                |
| <b>Project glossary</b>                 | Application-specific approved terminology                 | Supply or repair the target term without touching protected Reviewed/Approved exceptions. |
| <b>Contextual translation memory</b>    | Trusted prior segment in compatible context               | Suggest or reuse according to explicit confidence/status rules.                           |
| <b>Shared dictionary/calendar terms</b> | Vetted cross-project UI vocabulary                        | Repair common UI/calendar wording where locale support exists.                            |
| <b>Provider context</b>                 | Advisory description supplied with DeepL-capable requests | Improve disambiguation but never override structural validation.                          |
| <b>Provider draft</b>                   | Unreviewed external result                                | Populate eligible unresolved target text as machine origin.                               |

## 9. Provider Pipeline and Credential Security

TProviderSettings stores only provider, DeepL plan, remember policy, timeout, and batch size.

TProviderCredentialStore stores the selected secret in Windows Credential Manager or session memory.

TTranslationProviderClient builds and posts bounded HTTPS requests and maps provider-specific errors into ETranslationProviderError.

TContextBatching groups compatible contexts and limits request size. TPlaceholderProtection replaces immutable tokens before network transmission and requires exact recovery. TProviderRetry retries only transient failures with bounded backoff. TProviderLanguageCodes prevents sending a canonical locale code that a provider does not accept.

| Failure class                                 | Examples                                                           | Engineering response                                                                           |
|-----------------------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>Permanent authentication/configuration</b> | 401, 403, wrong DeepL plan, disabled Google API.                   | Fail promptly with actionable account/endpoint guidance; do not loop retries.                  |
| <b>Quota/rate</b>                             | 429.                                                               | Honor bounded retry policy and surface quota context.                                          |
| <b>Transient service/network</b>              | 5xx, temporary transport failure.                                  | Retry with bounded backoff and cancellation.                                                   |
| <b>Timeout</b>                                | Request exceeds configured seconds.                                | Cancel/abort the operation and return UI control; never leave the Wizard uncloseable.          |
| <b>Cancellation</b>                           | Operator stops final processing or closes within allowed boundary. | Raise controlled cancellation, preserve completed atomic artifacts, and report STOPPED safely. |
| <b>Structural response error</b>              | Wrong response count, malformed JSON, missing placeholders.        | Reject the batch; never write partial unvalidated translations as complete.                    |

## 10. Localization Review, Measurement, Layout, and Hyphenation

TLocalizationReviewer combines source geometry, code-geometry ownership, translated measurements, text roles, and language behavior into findings and proposals. ITextMeasurer isolates metric acquisition; FMX and GDI implementations provide framework-appropriate measurements.

Layout changes are stored as per-language rules and applied only after baseline restoration. A proposal is not application state until explicitly accepted. Code-positioned controls and complex layout relationships are excluded from automatic movement.

| Fitting order                 | Engineering rationale                                    | Stop condition                                   |
|-------------------------------|----------------------------------------------------------|--------------------------------------------------|
| <b>Natural space wrapping</b> | Preserves words, typography, and shared column geometry. | Text fits without clipping or harming neighbors. |

| Fitting order                            | Engineering rationale                                             | Stop condition                                                      |
|------------------------------------------|-------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Intelligent hyphenation</b>           | Breaks a long unspaced word at language-aware points.             | Readable result fits and remains linguistically acceptable.         |
| <b>Modest font reduction</b>             | Uses remaining control capacity without expanding shared layout.  | Do not cross the readable minimum or create inconsistent hierarchy. |
| <b>Conservative size/layout proposal</b> | Adds locale-specific room when ownership and neighbors permit.    | Reject if it moves code-owned controls or introduces collision.     |
| <b>Developer redesign</b>                | Handles complex collision, grid, graph, or container constraints. | Required when automated proposals cannot preserve the design.       |

## 11. Validation and Runtime Pack Export

TCatalogValidator separates blocking runtime safety from linguistic state. Required metadata, duplicate keys, source changes, placeholder/accelerator integrity, runtime wiring, and eligible target text are validated; Reviewed and Approved remain explicit human states.

TRuntimePackBuilder exports only after structural validation. It creates a compact pack with identity, framework, version/schema, source/target locale, checksum linkage, texts, templates, source maps needed for reversibility, and accepted layout rules.

- An error blocks export.
- A warning requires developer judgment and may describe linguistic or runtime-readiness risk.
- Information explains a condition that is safe but relevant.
- Canonical source and every target pack in one deployment must share compatible application identity and catalog/source contract.

## 12. Offline Runtime Manager and Pack Admission

TRuntimeLanguagePack parses one JSON file and exposes text, templates, source maps, locale, and layout rules. TTranslationRuntime discovers files, validates pack headers/checksums/framework/application identity, admits compatible packs, loads source and selected target, updates format settings, and serves translation lookups.

TLanguagePreference remembers an admitted locale. Startup loads the source pack as the safety anchor, then attempts the stored or system language according to policy. A stale preference cannot force an invalid pack into the process.

| Admission check                | Reason                                                                   |
|--------------------------------|--------------------------------------------------------------------------|
| ApplicationId                  | Prevents one product's pack from appearing in another.                   |
| Framework                      | Prevents incompatible layout/runtime assumptions between VCL and FMX.    |
| Schema/version                 | Prevents a runtime from misreading a newer/older contract.               |
| Locale/source locale           | Prevents mislabeled files and invalid fallback relationships.            |
| Checksum/source compatibility  | Prevents a translated pack from pairing with a different source catalog. |
| Required data/layout structure | Prevents incomplete pack activation and partial translation.             |

## 13. VCL Runtime Application and Lifecycle

TDATVCLLanguageManager connects TTranslationRuntime to VCL application lifecycle. Idle inspection, modal boundaries, active-form change, notifications, and explicit ApplyToForm cover existing and later forms without replacing application event handlers destructively.

TVCLTranslationApplicator snapshots source text, geometry, alignment, direction, and writable state; restores the baseline; applies stable-key text/templates/layout; handles menus, dialogs, status panels, lists/grids, wrapping, and RTL; then restores focus, selection, and other live state.

**VCL handle rule.** Do not force unnecessary handle recreation or overwrite window state merely to change language. Native controls and Windows messages can regenerate captions, so VCL template/caption interception reapplies only the owned text contract.

## 14. FMX Runtime Application and Lifecycle

TDATFMXLanguageManager connects TTranslationRuntime to FMX form lifecycle and styled-control behavior. It handles open/latent forms, tabs, scroll content bounds, browsers, grids, menus, dynamic refresh timers, and lifecycle-safe application without relying on one form-specific event.



TFMXTranslationApplicator snapshots/restores the designer baseline, applies text and layout to the FMX object tree, preserves state/order, and coordinates direction. Template rewrite is essential because FMX styles can recreate visible text after a property was translated.

**FMX styled-control rule.** A language switch is not complete merely because Text changed once. Styled controls, tab activation, browser document load, and scroll-layout recalculation can repaint later; lifecycle contracts must reapply only when the owned text is regenerated.

## 15. Components, Design Packages, and Designer Contract

DAT.Components.Core exposes published properties/events and the framework-neutral manager contract. DAT.Components.VCL/FMX implement framework lifecycle and applicator bridges. The LanguageSelector units provide connected designer combo boxes. Registration units expose those components only to RAD Studio design packages.

The design packages are Win32 IDE packages because RAD Studio is a Win32 design host even when the target application also builds Win64. Runtime packages are separated so a deployed application never links IDE registration code.

| Package                       | Role                               | Contains / requires                                                           |
|-------------------------------|------------------------------------|-------------------------------------------------------------------------------|
| DATLanguageManagerCoreRuntime | Framework-neutral runtime package. | Core runtime/manager/component contract used by VCL and FMX runtime packages. |
| DATLanguageManagerVCLRuntime  | VCL runtime package.               | VCL applicator, VCL manager, VCL selector, and framework-specific helpers.    |
| DATLanguageManagerFMXRuntime  | FMX runtime package.               | FMX applicator, FMX manager, FMX selector, and framework-specific helpers.    |
| DATLanguageManagerVCLDesign   | VCL IDE design package.            | Requires VCL runtime package and contains VCL Register.                       |
| DATLanguageManagerFMXDesign   | FMX IDE design package.            | Requires FMX runtime package and contains FMX Register.                       |

## 16. ComponentSource and Target dependencies

ComponentSource is a generated source-closure distribution for one target framework. The package generator computes the exact shared and framework-specific set and copies all of it into the kit. The target compiler must see one internally consistent version of that set.

The optional target-local dependencies folder is <Target>\dependencies\DelphiAppTranslation\source. It is not created automatically because the Studio cannot assume how a target project vendors third-party source or what repository policy permits.

| Layer                                 | Units                                                                                                             | Why inseparable                                                                                     |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Shared persistence/diagnostics</b> | DAT.Core.AtomicFile;<br>DAT.Core.Diagnostics                                                                      | Preference/layout/runtime parsing depends on the same atomic/error contract.                        |
| <b>Shared runtime</b>                 | DAT.Runtime.LanguagePack; Preference;<br>Manager; LayoutOverrides;<br>SplashTranslation; TemplateRewrite          | Pack model, active runtime, restoration, early startup, and later template repair share interfaces. |
| <b>Shared component API</b>           | DAT.Components.Core                                                                                               | VCL/FMX managers and application code compile against its published properties/events/functions.    |
| <b>VCL layer</b>                      | DAT.Runtime.VCL;<br>SplashTranslation.VCL;<br>TemplateRewrite.VCL;<br>DAT.Components.VCL;<br>VCL.LanguageSelector | All VCL lifecycle, applicator, splash, selector, and rewrite contracts must match.                  |
| <b>FMX layer</b>                      | DAT.Runtime.FMX;<br>SplashTranslation.FMX;<br>TemplateRewrite.FMX;<br>DAT.Components.FMX;<br>FMX.LanguageSelector | All FMX lifecycle, styles, selector, and rewrite contracts must match.                              |

**Partial-copy failure mode.** Copying only the units named by the first compiler errors can compile against older interfaces elsewhere on Search path, creating ambiguous unit resolution or runtime behavior. Refresh the complete generated set together.

## 17. Integration, Build, Deployment, and Project Boundaries

TComponentIntegrationPackageGenerator produces the recommended non-mutating kit.

TTargetBuildDeployer runs selected builds and deploys packs. TIntegrationPackageGenerator and TDelphiIntegrationSourceEditor retain an advanced explicitly authorized source-edit path with preview/backup/restore, but it is not the Wizard default.

Wizard final processing passes ComponentSource to its own build environment without persisting a DPROJ edit. Manual IDE builds require the developer to set Search path in Project Options. Deployment verifies the intended executable/application directory before copying packs.

- Search path must preserve existing values and \$(DCC\_UnitSearchPath).
- All configurations/all platforms is the normal setting when one vendored dependency folder serves every target.
- Pack deployment is independent of executable compilation; a build can succeed while packs are stale or absent, so both results must be checked.
- Unavailable optional removable/network destinations warn and skip; they must not fail an otherwise valid local build.
- No integration code should write outside the explicitly selected Studio export, workspace, target dependency, or authorized application destinations.

## 18. Studio Forms, State Machines, and Self-Localization

TfrmTranslationStudio is the landing and seven-page Maintenance state owner. TfrmSetupWizard is an eight-step validation state machine whose downstream state is invalidated when project/language/provider inputs change. TfrmLocalizationReview is modal within final processing and returns saved decisions to a continuation. DAT.Studio.Translation owns the Studio's own interface-language runtime.

The forms are designer-authored FMX resources. Runtime code should update state, enablement, text, and data; it should not rebuild the form hierarchy that belongs in the FMX designer. Default buttons, Enter/Esc behavior, hidden irrelevant buttons, and asynchronous provider state are explicit UI contracts.

| Form            | Primary state                                                                                                             | Critical transition                                                                                  |
|-----------------|---------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>MainForm</b> | Opened project, scan result, catalog, selected maintenance page, provider/settings, validation/export/integration status. | Landing to Wizard or Maintenance; language refresh must rebuild current visible content immediately. |

| Form                      | Primary state                                                                                                                       | Critical transition                                                                                            |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>SetupWizard</b>        | Current/highest step, project/profile, destinations, locales, provider, scan/catalog, authorization, final-processing continuation. | Begin Final Processing disables navigation; Localization Review temporarily suspends and resumes the pipeline. |
| <b>LocalizationReview</b> | Findings, glossary, suggestions, proposals, decisions, output package.                                                              | Close saves accepted decisions and returns to Wizard validation/export.                                        |

## 19. Directionality, HTML, Dynamic Text, and Reversible Layout

Text direction is pack locale metadata interpreted by the framework applicator. Direction and alignment are applied at the smallest correct paragraph/control ownership boundary; the whole interface is not blindly mirrored when the application contains protected LTR identifiers, paths, APIs, or numeric content.

Dynamic text uses stable semantic keys or source-map translation through `DATTranslateDynamicText`. HTML uses `DATTranslateHtmlText` and must rebuild the current page on language change. A language switch restores source/dynamic baseline before applying the new target, preventing Arabic text from surviving into Greek or another language.

| Content class                    | Translation contract                                                                    | Refresh contract                                                                            |
|----------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Designer property</b>         | Stable form.component.property key.                                                     | Applicator updates open form and lifecycle handles later forms.                             |
| <b>Resourcestring</b>            | Stable unit.symbol key plus explicit application use.                                   | Application code reads translated value at the intended message point.                      |
| <b>Dynamic status/message</b>    | Semantic key/template or source-map contract.                                           | Recompute immediately on language change without altering live data.                        |
| <b>HTML/report</b>               | Canonical HTML/text templates and semantic keys; table grid owns header/body alignment. | Rebuild current document synchronously on switch; no previous-language DOM/text may remain. |
| <b>Identifier/path/API token</b> | Protected LTR/nontranslatable token.                                                    | Preserve spelling and isolate direction inside RTL prose.                                   |

| Content class      | Translation contract                     | Refresh contract                                                 |
|--------------------|------------------------------------------|------------------------------------------------------------------|
| <b>Layout rule</b> | Locale/form/component property decision. | Restore designer baseline, then apply selected locale rule only. |

## 20. Diagnostics, Failure Containment, and Performance

TDATDiagnostics and structured exceptions provide severity, operation, path, provider status, stable key, and recovery context. UI code must convert them into actionable status/progress text without exposing secrets. Background provider/build work must always return the form to a closable and internally consistent state.

Performance-sensitive language switching must use already admitted in-memory packs. It should not rescan the project, call a provider, reread every JSON file repeatedly, or rebuild hidden pages that have no active content. Only open/later managed objects and explicit dynamic/HTML refresh consumers participate.

| Operation                   | Bound                                                                           | Cancellation / recovery                                                  |
|-----------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| <b>Provider request</b>     | Configured timeout (default 30 seconds) and bounded batch (default 40 strings). | Cancel check and retry policy; no endless wait.                          |
| <b>Build/deploy process</b> | Explicit process timeout and termination wait.                                  | Controlled process stop, captured output, destination-specific failure.  |
| <b>Scan</b>                 | Progress by stage/file and source snapshot.                                     | EProjectScanCancelled; prior catalog remains valid.                      |
| <b>Atomic save</b>          | One candidate validation and replace operation.                                 | Prior valid file and corruption quarantine.                              |
| <b>Language switch</b>      | Admitted pack lookup plus managed open objects.                                 | Reentrancy/main-thread guards and error behavior event/exception policy. |

## 21. Security and Privacy Engineering

- Secrets: Credential Manager/session memory only; never serialize or log API-key text.

- Network: development Studio only; HTTPS endpoints determined by selected provider/DeepL plan; response structure and placeholder count validated.
- Filesystem: normalize and validate intended paths; atomic writes; no broad recursive deletion; deployment only to verified application folders.
- Source: scanner parses text; it does not execute target Pascal, forms, scripts, or arbitrary JSON.
- Runtime: admit only compatible packs; reject malformed/wrong-app/wrong-framework/wrong-version/checksum mismatches before selection.
- Logs/reports: actionable identifiers and paths are acceptable; secrets and proprietary unneeded text are not.
- Distribution: exclude Local AppData workspaces, credentials, transient test folders, compiled output when source-only, and proprietary catalogs unless explicitly licensed.
- Dependencies: generated ComponentSource is source code and must be versioned/licensed/reviewed like any vendored library.

## 22. Test Architecture and Release Gates

The test suite is intentionally many small Delphi programs rather than one opaque test executable. Each program names and isolates a contract: scanning, placeholders, batching, retries, memory, validation, pack layout, VCL/FMX runtime, lifecycle, direction, wrapping, splash, templates, design streaming, and Studio form creation.

tools\verify\_all.ps1 is the release-level orchestration entry. Contract scripts compile and run groups using the RAD Studio environment. check\_shipped\_units\_complete.ps1 prevents package/kit/source-distribution omissions; check\_build\_paths\_agree.ps1 detects conflicting outputs; check\_source\_encoding.ps1 detects unsupported encoding drift.

| Gate           | Required evidence                                                                                         |
|----------------|-----------------------------------------------------------------------------------------------------------|
| Source hygiene | Encoding/path checks, no stale lock or unintended generated files, Git diff limited to approved scope.    |
| Compile        | Studio, package set, tools, and affected tests compile for intended platform/configuration.               |
| Foundation     | Types, JSON, workspace, atomic persistence, locale, placeholders, retry, terminology, memory, validation. |
| Scanner        | VCL/FMX form discovery, Pascal/resource contracts, counts, context, quality, source snapshot.             |

| Gate                 | Required evidence                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------|
| <b>Runtime</b>       | VCL/FMX load/apply/switch-back/restart, state preservation, lifecycle, templates, splash, wrapping, RTL.        |
| <b>Design-time</b>   | VCL/FMX form streaming and package registration with no missing-class or published-property error.              |
| <b>Studio</b>        | Main/Wizard/Review form creation, UI transitions, provider timeout/cancel, scan consistency, self-localization. |
| <b>Pilot matrix</b>  | Real target project, all supported languages, HTML/dynamic text, LTR/RTL, actual output folders.                |
| <b>Documentation</b> | DOCX source-driven accuracy, PDF generation, page rendering, visual inspection, accessibility/structure audits. |

## 23. Production Pascal Unit Reference

This chapter is the production PAS inventory. Each entry names its file, purpose, reason for separation, public surface, direct DAT dependencies, direct production consumers, named test consumers, and change risk. Dependencies are parsed from the current source; descriptions state the architectural responsibility rather than merely repeating the filename.

### 23.1 Core units

These units own framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction. The entries are ordered by unit name.

#### DAT.Core.AITranslation

**File:** source\core\DAT.Core.AITranslation.pas

**What it is:** Coordinates automatic translation at catalog level. It decides eligibility, invokes the provider client, preserves protected work, records origin/status, and produces review information instead of treating machine output as approved.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TAITranslationReview, TAITranslationWorkflow

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.CatalogJson, DAT.Core.Types

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 384 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

#### DAT.Core.AtomicFile

**File:** source\core\DAT.Core.AtomicFile.pas

**What it is:** Provides atomic UTF-8 persistence with validation, temporary files, previous-version recovery, corruption quarantine, and controlled replacement. Catalogs, packs, preferences, settings, glossaries, and deployment data rely on it to avoid partial JSON writes.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.



**Public surface:** TAtomicTextFile, EDATAAtomicFileError

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Core.AITranslation, DAT.Core.CatalogJson, DAT.Core.Glossary, DAT.Core.Hyphenation, DAT.Core.Interchange, DAT.Core.RuntimePack, DAT.Core.TranslationMemory, DAT.Integration.ComponentPackage, DAT.Integration.Package, DAT.Provider.Settings, DAT.Review.Localization, DAT.Runtime.LanguagePack, DAT.Runtime.LayoutOverrides, DAT.Runtime.Preference, DAT.Scan.DomainProfile, DAT.Studio.SetupWizard

**Named test consumers:** AtomicPersistenceSmokeTests, FMXDesignStreamingTests, FMXDialogTranslationSmokeTests, FMXLanguageManagerTests, FoundationSmokeTests, LanguageManagerCoreTests, VCLDesignStreamingTests, VCLLanguageManagerTests

**Change and validation risk:** The file contains 171 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.BuildInfo

**File:** source\core\DAT.Core.BuildInfo.pas

**What it is:** Supplies the compiled Studio build stamp and display description used in the title, Wizard header, reports, and diagnostics so artifacts can be traced to the producing build.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** StudioBuildStamp, StudioBuildDescription

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 61 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.CatalogJson

**File:** source\core\DAT.Core.CatalogJson.pas

**What it is:** Serializes and validates development catalogs and implements CSV import/export planning. It preserves stable keys, statuses, provenance, contexts, locale facts, and source checksums across round trips.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TCatalogCsvImportChange, TCatalogCsvImportPlan, TCatalogJson, TCatalogCsv, ECatalogJsonError, ECatalogCsvError

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Diagnostics, DAT.Core.Types, DAT.Scan.TextCodec

**Direct production consumers:** DAT.Core.AITranslation, DAT.Review.Localization, DAT.Scan.CatalogMerge, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, ProposalDecisionSmokeTests

**Change and validation risk:** The file contains 712 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.Diagnostics

**File:** source\core\DAT.Core.Diagnostics.pas

**What it is:** Centralizes structured information, warning, and error diagnostics so scanners, providers, integration, and runtime code report consistent severity and actionable context.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATDiagnosticSeverity, TDATDiagnostics

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Core.CatalogJson, DAT.Core.Glossary, DAT.Core.TranslationMemory, DAT.Provider.Settings, DAT.Runtime.LanguagePack, DAT.Runtime.LayoutOverrides, DAT.Runtime.Manager, DAT.Runtime.Preference

**Named test consumers:** FMXDesignStreamingTests, LanguageManagerCoreTests, VCLDesignStreamingTests

**Change and validation risk:** The file contains 111 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.Glossary

**File:** source\core\DAT.Core.Glossary.pas

**What it is:** Owns the project glossary model, matching rules, suggestions, catalog application, validation, and atomic persistence. It keeps product terminology explicit and reusable across provider runs.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TProjectGlossary, TProjectGlossaryTerm, TGlossarySuggestion, TProjectGlossarySuggester

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Diagnostics, DAT.Core.Types, DAT.Scan.TextCodec

**Direct production consumers:** DAT.Core.Interchange, DAT.Core.SharedDictionary, DAT.Provider.Glossary, DAT.Studio.LocalizationReview, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests, ProviderGlossarySmokeTests, TranslationMemorySmokeTests

**Change and validation risk:** The file contains 440 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.Hyphenation

**File:** source\core\DAT.Core.Hyphenation.pas

**What it is:** Loads language-aware hyphenation dictionaries and applies conservative word/text hyphenation. Runtime layout uses it only when ordinary wrapping and readable font fitting are insufficient.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATHyphenationDictionary, TDATHyphenation

**Direct DAT dependencies:** DAT.Core.AtomicFile

**Direct production consumers:** DAT.Core.RuntimePack, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests, HyphenationSmokeTests, PackLayoutSmokeTests

**Change and validation risk:** The file contains 411 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.Interchange

**File:** source\core\DAT.Core.Interchange.pas

**What it is:** Implements TMX translation-memory and TBX terminology interchange so reviewed material can move between the Studio and standards-aware localization tools.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTmxInterchange, TTbxInterchange

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Glossary, DAT.Core.TranslationMemory

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** TranslationMemorySmokeTests

**Change and validation risk:** The file contains 471 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.LocaleFacts

**File:** source\core\DAT.Core.LocaleFacts.pas

**What it is:** Reads Windows/Delphi locale facts such as native name, direction, date/time formats, decimal and thousands separators, and currency data for catalog metadata and runtime formatting.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TLocaleFacts, TLocaleFactsReader

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 135 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.ProjectDetection

**File:** source\core\DAT.Core.ProjectDetection.pas

**What it is:** Identifies a selected DPR/DPROJ, determines VCL or FMX, application ID, targets, forms, units, and output metadata, and rejects ambiguous or unsupported projects before scanning.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TProjectDetector, EProjectDetectionError

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 118 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.RuntimePack

**File:** source\core\DAT.Core.RuntimePack.pas

**What it is:** Builds the compact offline runtime JSON pack from a validated development catalog. It filters non-runtime records, carries locale/layout metadata, and writes checksum-compatible source and target packs.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TRuntimePackBuilder, ERuntimePackError

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Hyphenation, DAT.Core.Types, DAT.Runtime.LanguagePack, DAT.Validation.Catalog

**Direct production consumers:** DAT.Integration.ComponentPackage, DAT.Integration.Package, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests, HyphenationSmokeTests, PackLayoutSmokeTests

**Change and validation risk:** The file contains 415 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.SharedDictionary

**File:** source\core\DAT.Core.SharedDictionary.pas

**What it is:** Maintains the shared vetted wording dictionary used to repair or standardize recurring interface terms without overwriting reviewed or approved project-specific work.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TSharedDictionary

**Direct DAT dependencies:** DAT.Core.Glossary, DAT.Core.Types

**Direct production consumers:** DAT.Studio.SetupWizard

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 208 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.Terminology

**File:** source\core\DAT.Core.Terminology.pas

**What it is:** Resolves project glossary, shared dictionary, calendar terminology, protected tokens, and contextual wording in a deterministic priority order before or after provider translation.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTerminologyResolver

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 457 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.TranslationMemory

**File:** source\core\DAT.Core.TranslationMemory.pas

**What it is:** Stores and retrieves reviewed translations by exact, normalized, and similarity match. It preserves application and context provenance so later catalogs can reuse trusted wording without blind cross-key approval.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TMemoryMatchKind, TTranslationMemoryUnit, TMemoryMatch, TTranslationMemory, NormalizeSegment, SegmentSimilarity

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Diagnostics, DAT.Core.Types

**Direct production consumers:** DAT.Core.Interchange, DAT.Studio.SetupWizard

**Named test consumers:** TranslationMemorySmokeTests

**Change and validation risk:** The file contains 582 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.TranslationWorkspace

**File:** source\core\DAT.Core.TranslationWorkspace.pas

**What it is:** Defines the authoritative per-application workspace below Local AppData and returns paths for development catalogs, runtime packs, glossaries, deployment settings, and recovery artifacts.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTranslationWorkspace

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Integration.ComponentPackage, DAT.Integration.Package, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 123 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Core.Types

**File:** source\core\DAT.Core.Types.pas

**What it is:** Defines the central domain model: framework and status enumerations, project and locale profiles, translation entries, catalogs, ownership roles, and string conversions used by every pipeline stage.

**Why it matters:** This unit exists to keep framework-neutral catalogs, persistence, locale data, terminology, workspace ownership, and pack construction in the core layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTargetFramework, TTranslationStatus, TTranslationOrigin, TRuntimeApplicationKind, TRuntimeTextRole, TTextOwnershipKind, TProjectProfile, TLocaleProfile, TTranslationEntry, TTranslationCatalog, TargetFrameworkToString, StringToTargetFramework, TranslationStatusToString, StringToTranslationStatus; plus 16 additional public declarations

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Core.AITranslation, DAT.Core.CatalogJson, DAT.Core.Glossary, DAT.Core.ProjectDetection, DAT.Core.RuntimePack, DAT.Core.SharedDictionary, DAT.Core.Terminology,

DAT.Core.TranslationMemory, DAT.Core.TranslationWorkspace, DAT.Integration.ComponentPackage, DAT.Integration.MenuResource, DAT.Integration.Package, DAT.Review.ApplicationStrings, DAT.Review.Localization, DAT.Review.TextMeasurement, DAT.Review.TextMeasurement.FMX, DAT.Review.TextMeasurement.GDI, DAT.Scan.CatalogMerge, DAT.Scan.FormText, DAT.Scan.PascalResources, DAT.Scan.Project, DAT.Scan.Quality, DAT.Scan.Rules, DAT.Scan.Types, DAT.Studio.LocalizationReview, DAT.Studio.MainForm, DAT.Studio.SetupWizard, DAT.Validation.Catalog

**Named test consumers:** ApplicationStringSmokeTests, FormScanContracts, FoundationSmokeTests, HyphenationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, PackLayoutSmokeTests, ProposalDecisionSmokeTests, ProviderGlossarySmokeTests, ScannerSmokeTests, TranslationMemorySmokeTests

**Change and validation risk:** The file contains 579 lines. Changes require the core contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.2 Scan units

These units own project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging. The entries are ordered by unit name.

### DAT.Scan.CatalogMerge

**File:** source\scan\DAT.Scan.CatalogMerge.pas

**What it is:** Merges a new scan into an existing catalog by stable key and source text, detects collisions, preserves valid translations, recovers semantic contracts, and marks removed records obsolete.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TCatalogMergeSummary, TScanCatalogMerger, EScanKeyCollision

**Direct DAT dependencies:** DAT.Core.CatalogJson, DAT.Core.Types, DAT.Scan.Types

**Direct production consumers:** DAT.Integration.ComponentPackage, DAT.Integration.Package, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 440 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.



## DAT.Scan.Context

**File:** source\scan\DAT.Scan.Context.pas

**What it is:** Infers UI role, semantic concept, description, confidence, and surrounding context for each scanned string so provider output and human review are less ambiguous.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TScanContextAnalyzer

**Direct DAT dependencies:** DAT.Scan.DomainProfile, DAT.Scan.Types

**Direct production consumers:** DAT.Scan.FormText, DAT.Scan.PascalResources, DAT.Scan.Project

**Named test consumers:** ContextSmokeTests, FormScanContracts, FoundationSmokeTests, ScannerSmokeTests

**Change and validation risk:** The file contains 403 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.DomainProfile

**File:** source\scan\DAT.Scan.DomainProfile.pas

**What it is:** Builds an application-specific vocabulary and sense profile from project text, helping distinguish overloaded terms such as Play, Close, Schedule, and application-specific nouns.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TResolvedTerm, TApplicationDomainProfile, TDomainProfiler

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Scan.Types

**Direct production consumers:** DAT.Scan.Context, DAT.Studio.SetupWizard

**Named test consumers:** ContextSmokeTests, FormScanContracts, FoundationSmokeTests, ScannerSmokeTests

**Change and validation risk:** The file contains 625 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.FormText

**File:** source\scan\DAT.Scan.FormText.pas

**What it is:** Parses text DFM/FMX resources, walks component/property structure, applies eligibility rules, repairs encodings, and emits stable form.component.property scan items.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTextFormScanner

**Direct DAT dependencies:** DAT.Core.Types, DAT.Scan.Context, DAT.Scan.Quality, DAT.Scan.Rules, DAT.Scan.TextCodec, DAT.Scan.Types

**Direct production consumers:** DAT.Scan.Project

**Named test consumers:** FormScanContracts, FoundationSmokeTests

**Change and validation risk:** The file contains 405 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.PascalResources

**File:** source\scan\DAT.Scan.PascalResources.pas

**What it is:** Parses Pascal resourcestring declarations and approved runtime assignments while avoiding arbitrary code/data strings that cannot be safely localized.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TPascalResourceStringScanner

**Direct DAT dependencies:** DAT.Core.Types, DAT.Scan.Context, DAT.Scan.Quality, DAT.Scan.TextCodec, DAT.Scan.Types

**Direct production consumers:** DAT.Scan.Project

**Named test consumers:** FoundationSmokeTests, ScannerSmokeTests

**Change and validation risk:** The file contains 1,765 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.Project

**File:** source\scan\DAT.Scan.Project.pas

**What it is:** Orchestrates complete project scanning across detected form and source files, progress/cancellation, context analysis, quality checks, and result provenance.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TProjectScanner

**Direct DAT dependencies:** DAT.Core.Types, DAT.Scan.Context, DAT.Scan.FormText, DAT.Scan.PascalResources, DAT.Scan.Quality, DAT.Scan.TextCodec, DAT.Scan.Types

**Direct production consumers:** DAT.Integration.ComponentPackage, DAT.Integration.Package, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 671 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.Quality

**File:** source\scan\DAT.Scan.Quality.pas

**What it is:** Analyzes scan output for suspicious keys, duplicate occurrences, encoding problems, dynamic/runtime ownership, missing context, and other conditions requiring operator attention.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TScanQualityAnalyzer

**Direct DAT dependencies:** DAT.Core.Types, DAT.Scan.Types

**Direct production consumers:** DAT.Scan.FormText, DAT.Scan.PascalResources, DAT.Scan.Project

**Named test consumers:** FormScanContracts, FoundationSmokeTests, ScannerSmokeTests

**Change and validation risk:** The file contains 106 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.Rules

**File:** source\scan\DAT.Scan.Rules.pas

**What it is:** Centralizes the universal include/exclude/property rules used by both VCL and FMX scanning so a project is not governed by application-specific exceptions.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TScanRuleSet

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Scan.FormText

**Named test consumers:** FormScanContracts, FoundationSmokeTests, ScannerSmokeTests

**Change and validation risk:** The file contains 188 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.TextCodec

**File:** source\scan\DAT.Scan.TextCodec.pas

**What it is:** Loads Delphi text safely across BOM/code-page variants, repairs known mojibake, and decodes Delphi string expressions without executing source code.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** LoadDelphiTextFile, RepairMisdecodedText, TryDecodeDelphiStringExpression

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Core.CatalogJson, DAT.Core.Glossary, DAT.Integration.MenuResource, DAT.Review.Localization, DAT.Scan.FormText, DAT.Scan.PascalResources, DAT.Scan.Project

**Named test consumers:** FormScanContracts, FoundationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, ProposalDecisionSmokeTests, ScannerSmokeTests

**Change and validation risk:** The file contains 210 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Scan.Types

**File:** source\scan\DAT.Scan.Types.pas

**What it is:** Defines scan items, diagnostics, kinds, progress/cancellation callbacks, source snapshots, counts, and the result object passed into catalog merge and UI reporting.

**Why it matters:** This unit exists to keep project discovery, DFM/FMX and Pascal text extraction, context, rules, quality, and stable-key catalog merging in the scan layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TScannedTextKind, TScanDiagnosticSeverity, TScanItem, TScanDiagnostic, TProjectScanResult, EProjectScanCancelled, ScannedTextKindDisplayName

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Integration.ComponentPackage, DAT.Integration.Package, DAT.Scan.CatalogMerge, DAT.Scan.Context, DAT.Scan.DomainProfile, DAT.Scan.FormText, DAT.Scan.PascalResources, DAT.Scan.Project, DAT.Scan.Quality, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** ContextSmokeTests, FormScanContracts, FoundationSmokeTests, ScannerSmokeTests

**Change and validation risk:** The file contains 183 lines. Changes require the scan contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.3 Provider units

These units own DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation. The entries are ordered by unit name.

### DAT.Provider.Batching

**File:** source\provider\DAT.Provider.Batching.pas

**What it is:** Groups entries by compatible context and splits them into bounded request batches so provider limits, latency, cancellation, and context quality remain predictable.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TContextGroup, TContextBatching

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Provider.Client

**Named test consumers:** ContextBatchingSmokeTests, FoundationSmokeTests

**Change and validation risk:** The file contains 122 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Provider.CalendarTerms**

**File:** source\provider\DAT.Provider.CalendarTerms.pas

**What it is:** Supplies locale-specific calendar vocabulary and protects weekday/month meanings from ambiguous short-word provider output.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TCalendarTerms

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Provider.Client

**Named test consumers:** CalendarTermSmokeTests, FoundationSmokeTests

**Change and validation risk:** The file contains 285 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Provider.Client**

**File:** source\provider\DAT.Provider.Client.pas

**What it is:** Implements the bounded HTTPS client for DeepL API Free/Pro and Google Cloud Translation Basic v2, including request bodies, response parsing, contexts, cancellation, timeout, retries, and normalized errors.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTranslationCancelled, TTranslationProviderClient

**Direct DAT dependencies:** DAT.Provider.Batching, DAT.Provider.CalendarTerms, DAT.Provider.LanguageCodes, DAT.Provider.Placeholders, DAT.Provider.Retry, DAT.Provider.Types

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 600 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Provider.CredentialStore

**File:** source\provider\DAT.Provider.CredentialStore.pas

**What it is:** Stores, retrieves, and removes provider secrets as Windows Generic Credentials. It deliberately separates secrets from provider-settings JSON, catalogs, packs, source, logs, and exports.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TProviderCredentialStore, ECredentialStoreError

**Direct DAT dependencies:** DAT.Provider.Types

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 166 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Provider.Glossary

**File:** source\provider\DAT.Provider.Glossary.pas

**What it is:** Converts vetted project terminology into provider-facing glossary behavior where supported and local pre/post-processing where the provider API does not offer a glossary feature.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TGlossaryEntry, TProviderGlossary

**Direct DAT dependencies:** DAT.Core.Glossary

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** ProviderGlossarySmokeTests

**Change and validation risk:** The file contains 230 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Provider.LanguageCodes

**File:** source\provider\DAT.Provider.LanguageCodes.pas

**What it is:** Maps canonical locale codes to the distinct source/target language codes accepted by DeepL and Google, including provider-specific aliases and unsupported-locale checks.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TProviderLanguageCodes

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Provider.Client

**Named test consumers:** FoundationSmokeTests, LanguageCodeSmokeTests

**Change and validation risk:** The file contains 86 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Provider.Placeholders

**File:** source\provider\DAT.Provider.Placeholders.pas

**What it is:** Protects Delphi format placeholders, accelerators, markup, identifiers, and other immutable tokens before a provider call and verifies exact restoration afterward.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TPlaceholderProtection

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Provider.Client

**Named test consumers:** FoundationSmokeTests, PlaceholderSmokeTests

**Change and validation risk:** The file contains 239 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Provider.Retry

**File:** source\provider\DAT.Provider.Retry.pas

**What it is:** Defines bounded retry policy for transient provider and network failures, including backoff and retryable status classification without retrying permanent authentication errors indefinitely.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TProviderRetry

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.



**Direct production consumers:** DAT.Provider.Client

**Named test consumers:** FoundationSmokeTests, RetrySmokeTests

**Change and validation risk:** The file contains 92 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Provider.Settings

**File:** source\provider\DAT.Provider.Settings.pas

**What it is:** Owns non-secret provider configuration under Local AppData: selected provider, DeepL plan, remember policy, timeout, and batch size. API key text is intentionally excluded.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TProviderSettings

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Diagnostics, DAT.Provider.Types

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 141 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Provider.Types

**File:** source\provider\DAT.Provider.Types.pas

**What it is:** Defines provider and DeepL-plan enumerations, display/string conversions, and the structured provider exception carrying status and retry context.

**Why it matters:** This unit exists to keep DeepL/Google configuration, credential protection, batching, placeholders, retries, and HTTPS translation in the provider layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTranslationProvider, TDeepLPlan, ETranslationProviderError, TranslationProviderDisplayName, TranslationProviderToString, StringToTranslationProvider, DeepLPlanToString, StringToDeepLPlan

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Provider.Client, DAT.Provider.CredentialStore, DAT.Provider.Settings, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 93 lines. Changes require the provider contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.4 Review units

These units own localization findings, layout proposals, text measurement, and application-owned string review. The entries are ordered by unit name.

### DAT.Review.ApplicationStrings

**File:** source\review\DAT.Review.ApplicationStrings.pas

**What it is:** Classifies strings owned by the application rather than by static form resources, providing focused review of live status, generated messages, templates, and runtime text contracts.

**Why it matters:** This unit exists to keep localization findings, layout proposals, text measurement, and application-owned string review in the review layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TApplicationOwnedStrings

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Studio.LocalizationReview

**Named test consumers:** ApplicationStringSmokeTests, FoundationSmokeTests

**Change and validation risk:** The file contains 164 lines. Changes require the review contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Review.CodeGeometry

**File:** source\review\DAT.Review.CodeGeometry.pas

**What it is:** Extracts control geometry assignments from Pascal so layout review can distinguish designer-owned positions from deliberately code-positioned controls that must not be moved automatically.

**Why it matters:** This unit exists to keep localization findings, layout proposals, text measurement, and application-owned string review in the review layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TCodeGeometry

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** DAT.Review.Localization

**Named test consumers:** FoundationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, ProposalDecisionSmokeTests

**Change and validation risk:** The file contains 233 lines. Changes require the review contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Review.Localization

**File:** source\review\DAT.Review.Localization.pas

**What it is:** Runs linguistic and layout review, creates findings and conservative per-language layout proposals, records accept/reject decisions, and prepares the HTML/JSON review package.

**Why it matters:** This unit exists to keep localization findings, layout proposals, text measurement, and application-owned string review in the review layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TLocalizationFindingSeverity, TLocalizationFinding, TLayoutControl, TLayoutProposal, TLocalizationReview, TLocalizationReviewer, LocalizationFindingSeverityText

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.CatalogJson, DAT.Core.Types, DAT.Review.CodeGeometry, DAT.Review.TextMeasurement, DAT.Runtime.LanguagePack, DAT.Scan.TextCodec

**Direct production consumers:** DAT.Studio.LocalizationReview, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, ProposalDecisionSmokeTests

**Change and validation risk:** The file contains 5,657 lines. Changes require the review contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Review.TextMeasurement

**File:** source\review\DAT.Review.TextMeasurement.pas

**What it is:** Defines the platform-neutral text-measurement interface and fitting calculations used to compare translated text against available control geometry.

**Why it matters:** This unit exists to keep localization findings, layout proposals, text measurement, and application-owned string review in the review layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTextMeasurement

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Review.Localization, DAT.Review.TextMeasurement.FMX, DAT.Review.TextMeasurement.GDI

**Named test consumers:** FoundationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, ProposalDecisionSmokeTests

**Change and validation risk:** The file contains 89 lines. Changes require the review contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Review.TextMeasurement.FMX**

**File:** source\review\DAT.Review.TextMeasurement.FMX.pas

**What it is:** Provides FMX canvas-based text measurement so font, wrapping, and control-size proposals reflect FireMonkey rendering characteristics.

**Why it matters:** This unit exists to keep localization findings, layout proposals, text measurement, and application-owned string review in the review layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** No separate public type or routine; the unit supplies registrations or implementation bindings.

**Direct DAT dependencies:** DAT.Core.Types, DAT.Review.TextMeasurement

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** FoundationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, ProposalDecisionSmokeTests

**Change and validation risk:** The file contains 57 lines. Changes require the review contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Review.TextMeasurement.GDI**

**File:** source\review\DAT.Review.TextMeasurement.GDI.pas

**What it is:** Provides Windows GDI text measurement for VCL controls and report/header fitting using the actual font metrics available on Windows.

**Why it matters:** This unit exists to keep localization findings, layout proposals, text measurement, and application-owned string review in the review layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** No separate public type or routine; the unit supplies registrations or implementation bindings.

**Direct DAT dependencies:** DAT.Core.Types, DAT.Review.TextMeasurement

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** FoundationSmokeTests, LayoutContracts, LayoutFittingSmokeTests, ProposalDecisionSmokeTests

**Change and validation risk:** The file contains 124 lines. Changes require the review contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.5 Validation units

These units own structural catalog checks that protect runtime export. The entries are ordered by unit name.

### DAT.Validation.Catalog

**File:** source\validation\DAT.Validation.Catalog.pas

**What it is:** Performs blocking structural validation and non-blocking review warnings for locale metadata, required text, placeholders, accelerators, duplicate keys, source changes, runtime wiring, and export readiness.

**Why it matters:** This unit exists to keep structural catalog checks that protect runtime export in the validation layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TValidationSeverity, TValidationIssue, TCatalogValidationResult, TCatalogValidator, ValidationSeverityDisplayName

**Direct DAT dependencies:** DAT.Core.Types

**Direct production consumers:** DAT.Core.RuntimePack, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 403 lines. Changes require the validation contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.6 Runtime units

These units own offline pack loading, preference, translation application, layout restoration, templates, and splash handling. The entries are ordered by unit name.

### DAT.Runtime.FMX

**File:** source\runtime\DAT.Runtime.FMX.pas

**What it is:** Applies runtime packs to FMX object trees while preserving live state, snapshots, designer geometry, control order, styled controls, grids, menus, scrolling, wrapping, and language direction.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** No separate public type or routine; the unit supplies registrations or implementation bindings.

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.LayoutOverrides

**Direct production consumers:** DAT.Components.FMX, DAT.Components.FMX.Spike, DAT.Runtime.SplashTranslation.FMX, DAT.Studio.Translation

**Named test consumers:** FMXDesignStreamingTests, FMXDialogTranslationSmokeTests, FMXDiscoverySmokeTests, FMXGridOrderSmokeTests, FMXLanguageManagerTests, FMXManagerLifecycleSpikeTests, FMXMenuOrderSmokeTests, FMXRightToLeftSmokeTests, FMXRuntimeSmokeTests, FMXSplashHookSmokeTests, FMXStyledRuntimeSmokeTests, FMXWrapSmokeTests

**Change and validation risk:** The file contains 3,058 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Runtime.LanguagePack

**File:** source\runtime\DAT.Runtime.LanguagePack.pas

**What it is:** Parses and validates deployed JSON packs, indexes stable keys/source text/templates/layout rules, supports dynamic and HTML translation, and exposes locale and compatibility metadata to the runtime manager.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TRuntimeLocale, TRuntimeLayoutRule, TRuntimeLanguagePack, TLanguagePackDescriptor, TLanguagePackDiscovery, ELanguagePackError, CanonicalNativeLanguageName, IsRuntimeLayoutProperty, IsAutomaticallySafeLayoutProperty

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Diagnostics

**Direct production consumers:** DAT.Components.Core, DAT.Components.FMX, DAT.Components.FMX.LanguageSelector, DAT.Components.FMX.Spike, DAT.Components.VCL, DAT.Components.VCL.LanguageSelector, DAT.Components.VCL.Spike, DAT.Core.RuntimePack, DAT.Integration.BuildDeploy, DAT.Integration.ComponentPackage, DAT.Integration.MenuResource, DAT.Integration.Package, DAT.Review.Localization, DAT.Runtime.FMX, DAT.Runtime.Manager, DAT.Runtime.SplashTranslation, DAT.Runtime.SplashTranslation.FMX, DAT.Runtime.SplashTranslation.VCL, DAT.Runtime.TemplateRewrite, DAT.Runtime.TemplateRewrite.FMX, DAT.Runtime.TemplateRewrite.VCL, DAT.Runtime.VCL, DAT.Studio.LocalizationReview, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FMXCaptionRewriteSmokeTests, FMXDesignStreamingTests, FMXDialogTranslationSmokeTests, FMXDiscoverySmokeTests, FMXGridOrderSmokeTests,

FMXLanguageManagerTests, FMXManagerLifecycleSpikeTests, FMXMenuOrderSmokeTests, FMXRightToLeftSmokeTests, FMXRuntimeSmokeTests, FMXSplashHookSmokeTests, FMXStyledRuntimeSmokeTests, FMXWrapSmokeTests, FoundationSmokeTests, LanguageManagerCoreTests, LayoutOverrideSmokeTests, TemplateRewriteSmokeTests, VCLCaptionInterceptSmokeTests, VCLDesignStreamingTests, VCLDiscoverySmokeTests, VCLLanguageManagerTests, VCLManagerLifecycleSpikeTests, VCLManagerMDILifecycleSpikeTests, VCLRightToLeftSmokeTests, VCLRuntimeSmokeTests, VCLSplashHookSmokeTests, VCLSplashSmokeTests, VCLStatusPanelInterceptTests, VCLWrapSmokeTests

**Change and validation risk:** The file contains 1,305 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Runtime.LayoutOverrides

**File:** source\runtime\DAT.Runtime.LayoutOverrides.pas

**What it is:** Persists developer-entered per-language control overrides separately from generated packs so hand-tuned positions, sizes, and font choices remain explicit and recoverable.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TLayoutOverride, TLayoutOverrides

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Diagnostics

**Direct production consumers:** DAT.Runtime.FMX, DAT.Runtime.VCL

**Named test consumers:** FMXStyledRuntimeSmokeTests, LayoutOverrideSmokeTests

**Change and validation risk:** The file contains 399 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Runtime.Manager

**File:** source\runtime\DAT.Runtime.Manager.pas

**What it is:** Owns offline pack discovery, admission, checksum/source-pack compatibility, selected locale, formatting settings, translation lookup, preference fallback, and the active source/target runtime pair.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTranslationRuntime

**Direct DAT dependencies:** DAT.Core.Diagnostics, DAT.Runtime.LanguagePack, DAT.Runtime.Preference

**Direct production consumers:** DAT.Components.Core, DAT.Integration.Package, DAT.Studio.Translation

**Named test consumers:** FMXDesignStreamingTests, FMXDialogTranslationSmokeTests, FMXDiscoverySmokeTests, FMXLanguageManagerTests, FoundationSmokeTests, LanguageManagerCoreTests, VCLDesignStreamingTests, VCLLanguageManagerTests, VCLSplashSmokeTests

**Change and validation risk:** The file contains 636 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Runtime.Preference

**File:** source\runtime\DAT.Runtime.Preference.pas

**What it is:** Reads and atomically writes the selected language code in the configured Local AppData or executable-folder preference location, with safe fallback from stale or invalid values.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TLanguagePreference

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.Diagnostics

**Direct production consumers:** DAT.Runtime.Manager, DAT.Runtime.SplashTranslation

**Named test consumers:** FMXDesignStreamingTests, FMXDialogTranslationSmokeTests, FMXDiscoverySmokeTests, FMXLanguageManagerTests, FMXSplashHookSmokeTests, FoundationSmokeTests, LanguageManagerCoreTests, VCLDesignStreamingTests, VCLLanguageManagerTests, VCLSplashHookSmokeTests, VCLSplashSmokeTests

**Change and validation risk:** The file contains 105 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Runtime.SplashTranslation

**File:** source\runtime\DAT.Runtime.SplashTranslation.pas

**What it is:** Defines the framework-neutral splash translation contract and keys so early startup text can be translated before the main form and standard lifecycle hooks exist.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATSplashTranslation

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.Preference



**Direct production consumers:** DAT.Components.FMX, DAT.Components.VCL, DAT.Runtime.SplashTranslation.FMX, DAT.Runtime.SplashTranslation.VCL

**Named test consumers:** FMXSplashHookSmokeTests, VCLSplashHookSmokeTests

**Change and validation risk:** The file contains 176 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Runtime.SplashTranslation.FMX**

**File:** source\runtime\DAT.Runtime.SplashTranslation.FMX.pas

**What it is:** Implements early FMX splash translation and coordinates the selected runtime before ordinary FMX form discovery begins.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATAFMXSplashTranslation

**Direct DAT dependencies:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.SplashTranslation

**Direct production consumers:** DAT.Components.FMX

**Named test consumers:** FMXSplashHookSmokeTests

**Change and validation risk:** The file contains 110 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Runtime.SplashTranslation.VCL**

**File:** source\runtime\DAT.Runtime.SplashTranslation.VCL.pas

**What it is:** Implements early VCL splash translation while respecting VCL handle creation and startup ordering.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATAVCLSplashTranslation

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.SplashTranslation, DAT.Runtime.VCL

**Direct production consumers:** DAT.Components.VCL

**Named test consumers:** VCLSplashHookSmokeTests

**Change and validation risk:** The file contains 92 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Runtime.TemplateRewrite**

**File:** source\runtime\DAT.Runtime.TemplateRewrite.pas

**What it is:** Provides the framework-neutral contract for translating templated or owner-drawn text whose final string is produced after normal property translation.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATTemplateRewriter

**Direct DAT dependencies:** DAT.Runtime.LanguagePack

**Direct production consumers:** DAT.Runtime.TemplateRewrite.FMX, DAT.Runtime.TemplateRewrite.VCL, DAT.Runtime.VCL

**Named test consumers:** FMXCaptionRewriteSmokeTests, TemplateRewriteSmokeTests, VCLCaptionInterceptSmokeTests

**Change and validation risk:** The file contains 339 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Runtime.TemplateRewrite.FMX**

**File:** source\runtime\DAT.Runtime.TemplateRewrite.FMX.pas

**What it is:** Applies FMX-specific template and caption rewrites after styles or control templates regenerate visible text.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDAFMXTemplateRewrite

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.TemplateRewrite

**Direct production consumers:** DAT.Components.FMX

**Named test consumers:** FMXCaptionRewriteSmokeTests

**Change and validation risk:** The file contains 74 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Runtime.TemplateRewrite.VCL

**File:** source\runtime\DAT.Runtime.TemplateRewrite.VCL.pas

**What it is:** Intercepts and reapplies VCL caption/template text that Windows messages or owner-drawn controls can recreate after the initial translation pass.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATVCLTemplateIntercept

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.TemplateRewrite

**Direct production consumers:** DAT.Components.VCL

**Named test consumers:** VCLCaptionInterceptSmokeTests

**Change and validation risk:** The file contains 149 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Runtime.VCL

**File:** source\runtime\DAT.Runtime.VCL.pas

**What it is:** Applies packs to VCL windows and controls while preserving handles, focus, selections, list/grid state, designer layout, status panels, menus, dialogs, wrapping, and reversible RTL state.

**Why it matters:** This unit exists to keep offline pack loading, preference, translation application, layout restoration, templates, and splash handling in the runtime layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** No separate public type or routine; the unit supplies registrations or implementation bindings.

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.LayoutOverrides, DAT.Runtime.TemplateRewrite

**Direct production consumers:** DAT.Components.VCL, DAT.Components.VCL.Spike, DAT.Runtime.SplashTranslation.VCL

**Named test consumers:** LayoutOverrideSmokeTests, VCLDesignStreamingTests, VCLDiscoverySmokeTests, VCLLanguageManagerTests, VCLManagerLifecycleSpikeTests, VCLManagerMDILifecycleSpikeTests, VCLRightToLeftSmokeTests, VCLRuntimeSmokeTests, VCLSplashHookSmokeTests, VCLSplashSmokeTests, VCLStatusPanelInterceptTests, VCLWrapSmokeTests

**Change and validation risk:** The file contains 2,775 lines. Changes require the runtime contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.7 Components units

These units own designer-visible language-manager components, lifecycle adapters, and language selectors. The entries are ordered by unit name.

### DAT.Components.Core

**File:** source\components\DAT.Components.Core.pas

**What it is:** Defines the framework-neutral component contract shared by VCL and FMX managers: published configuration, language-change events, form identity mapping, error policy, and the process-wide translation functions used by dynamic text and HTML producers.

**Why it matters:** This unit exists to keep designer-visible language-manager components, lifecycle adapters, and language selectors in the components layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATPreferenceLocation, DATMissingPackBehavior, DATErrorBehavior, DATCustomLanguageManager, EDATLanguageManagerError, EDATLanguageManagerConfigurationError, EDATLanguageManagerMainThreadError, EDATLanguageManagerReentrancyError, EDATFormIdentityError, DATTranslateText, DATFormatText, DATActiveLanguageCode, DATActiveTextDirection, DATTranslateDynamicText; plus 1 additional public declarations

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.Manager

**Direct production consumers:** DAT.Components.FMX, DAT.Components.VCL

**Named test consumers:** FMXDesignStreamingTests, FMXDialogTranslationSmokeTests, FMXDiscoverySmokeTests, FMXLanguageManagerTests, LanguageManagerCoreTests, VCLDesignStreamingTests, VCLDiscoverySmokeTests, VCLLanguageManagerTests, VCLSplashSmokeTests

**Change and validation risk:** The file contains 1,221 lines. Changes require the components contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Components.FMX

**File:** source\components\DAT.Components.FMX.pas

**What it is:** Implements the production FMX language manager and its application lifecycle integration. It discovers forms, preserves state and designer geometry, applies packs, refreshes tabs, scroll boxes, browsers, grids, and dynamic content, and enforces reversible LTR/RTL behavior.

**Why it matters:** This unit exists to keep designer-visible language-manager components, lifecycle adapters, and language selectors in the components layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATScrollBoundsSubscription, TDATBrowserLifecycleSubscription, TDATTabChangeSubscription, TDATFMXLanguageManager

**Direct DAT dependencies:** DAT.Components.Core, DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.SplashTranslation, DAT.Runtime.SplashTranslation.FMX, DAT.Runtime.TemplateRewrite.FMX

**Direct production consumers:** DAT.Components.FMX.LanguageSelector, DAT.Design.FMX.Register

**Named test consumers:** FMXDesignStreamingTests, FMXDialogTranslationSmokeTests, FMXDiscoverySmokeTests, FMXLanguageManagerTests

**Change and validation risk:** The file contains 2,275 lines. Changes require the components contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Components.FMX.LanguageSelector

**File:** source\components\DAT.Components.FMX.LanguageSelector.pas

**What it is:** Implements the FMX TDATFMXLanguageComboBox. It binds to one FMX language manager, lists admitted packs, optionally shows locale codes, and changes language without duplicating manager logic.

**Why it matters:** This unit exists to keep designer-visible language-manager components, lifecycle adapters, and language selectors in the components layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATFMXLanguageComboBox

**Direct DAT dependencies:** DAT.Components.FMX, DAT.Runtime.LanguagePack

**Direct production consumers:** DAT.Design.FMX.Register

**Named test consumers:** FMXDesignStreamingTests

**Change and validation risk:** The file contains 170 lines. Changes require the components contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Components.FMX.Spike

**File:** source\components\DAT.Components.FMX.Spike.pas

**What it is:** Retains the focused FMX lifecycle spike used to prove form discovery and exactly-once application behavior before those contracts were promoted into the production FMX manager.

**Why it matters:** This unit exists to keep designer-visible language-manager components, lifecycle adapters, and language selectors in the components layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATFMXLanguageManagerSpike

**Direct DAT dependencies:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** FMXManagerLifecycleSpikeTests

**Change and validation risk:** The file contains 168 lines. Changes require the components contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Components.VCL

**File:** source\components\DAT.Components.VCL.pas

**What it is:** Implements the production VCL language manager. It observes application idle, modal, and active-form transitions; translates open and later forms; preserves live control state; restores source geometry; and applies VCL-specific RTL and layout contracts.

**Why it matters:** This unit exists to keep designer-visible language-manager components, lifecycle adapters, and language selectors in the components layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATVCLLanguageManager

**Direct DAT dependencies:** DAT.Components.Core, DAT.Runtime.LanguagePack, DAT.Runtime.SplashTranslation, DAT.Runtime.SplashTranslation.VCL, DAT.Runtime.TemplateRewrite.VCL, DAT.Runtime.VCL

**Direct production consumers:** DAT.Components.VCL.LanguageSelector, DAT.Design.VCL.Register

**Named test consumers:** VCLDesignStreamingTests, VCLDiscoverySmokeTests, VCLLanguageManagerTests, VCLSplashSmokeTests

**Change and validation risk:** The file contains 352 lines. Changes require the components contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Components.VCL.LanguageSelector

**File:** source\components\DAT.Components.VCL.LanguageSelector.pas

**What it is:** Implements the VCL TDATVCLLanguageComboBox. It connects through a published LanguageManager property, refreshes available packs, and selects the locale while remaining fully designer-editable.

**Why it matters:** This unit exists to keep designer-visible language-manager components, lifecycle adapters, and language selectors in the components layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATVCLLanguageComboBox

**Direct DAT dependencies:** DAT.Components.VCL, DAT.Runtime.LanguagePack

**Direct production consumers:** DAT.Design.VCL.Register

**Named test consumers:** VCLDesignStreamingTests

**Change and validation risk:** The file contains 170 lines. Changes require the components contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Components.VCL.Spike

**File:** source\components\DAT.Components.VCL.Spike.pas

**What it is:** Retains the VCL lifecycle proof harness that established idle discovery, notification cleanup, and exactly-once form application before the production VCL component was finalized.

**Why it matters:** This unit exists to keep designer-visible language-manager components, lifecycle adapters, and language selectors in the components layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDATVCLLanguageManagerSpike

**Direct DAT dependencies:** DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** VCLManagerLifecycleSpikeTests, VCLManagerMDILifecycleSpikeTests

**Change and validation risk:** The file contains 123 lines. Changes require the components contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.8 Design units

These units own RAD Studio Tool Palette registration and design-time package exposure. The entries are ordered by unit name.

### DAT.Design.FMX.Register

**File:** source\design\DAT.Design.FMX.Register.pas

**What it is:** Registers the FMX language manager and selector on the DAT Localization Tool Palette page. It is compiled only into the FMX design-time package, never into a deployed application.

**Why it matters:** This unit exists to keep RAD Studio Tool Palette registration and design-time package exposure in the design layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** Register

**Direct DAT dependencies:** DAT.Components.FMX, DAT.Components.FMX.LanguageSelector

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 20 lines. Changes require the design contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Design.VCL.Register**

**File:** source\design\DAT.Design.VCL.Register.pas

**What it is:** Registers the VCL language manager and selector on the DAT Localization Tool Palette page. It is the VCL design-time bridge between package installation and Object Inspector use.

**Why it matters:** This unit exists to keep RAD Studio Tool Palette registration and design-time package exposure in the design layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** Register

**Direct DAT dependencies:** DAT.Components.VCL, DAT.Components.VCL.LanguageSelector

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 20 lines. Changes require the design contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## **23.9 Integration units**

These units own component-kit generation, controlled Delphi project integration, build, and deployment. The entries are ordered by unit name.

### **DAT.Integration.BuildDeploy**

**File:** source\integration\DAT.Integration.BuildDeploy.pas

**What it is:** Builds selected Delphi platform/configuration targets and deploys matching packs to detected or authorized application folders while reporting command, output, timeout, and destination results.

**Why it matters:** This unit exists to keep component-kit generation, controlled Delphi project integration, build, and deployment in the integration layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TTargetBuildDeployer

**Direct DAT dependencies:** DAT.Runtime.LanguagePack



**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 698 lines. Changes require the integration contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Integration.ComponentPackage

**File:** source\integration\DAT.Integration.ComponentPackage.pas

**What it is:** Generates the recommended non-mutating component integration kit: the complete framework-appropriate ComponentSource set, packs, manifest, README, deployment script, and completion report.

**Why it matters:** This unit exists to keep component-kit generation, controlled Delphi project integration, build, and deployment in the integration layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TComponentIntegrationPackageGenerator

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.RuntimePack, DAT.Core.TranslationWorkspace, DAT.Core.Types, DAT.Runtime.LanguagePack, DAT.Scan.CatalogMerge, DAT.Scan.Project, DAT.Scan.Types

**Direct production consumers:** DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 614 lines. Changes require the integration contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Integration.DelphiSource

**File:** source\integration\DAT.Integration.DelphiSource.pas

**What it is:** Implements the explicitly advanced source-integration editor with preview, authorization, transaction backup, bounded changes, and restoration; it is not used by the recommended component workflow.

**Why it matters:** This unit exists to keep component-kit generation, controlled Delphi project integration, build, and deployment in the integration layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TDelphiIntegrationSourceEditor

**Direct DAT dependencies:** No DAT unit dependency; this is a leaf/foundation unit.

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 536 lines. Changes require the integration contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Integration.MenuResource**

**File:** source\integration\DAT.Integration.MenuResource.pas

**What it is:** Plans and applies controlled language-menu resource changes for advanced integration, including safe detection and reversal of the generated menu block.

**Why it matters:** This unit exists to keep component-kit generation, controlled Delphi project integration, build, and deployment in the integration layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TMenuResourceEdit, TLanguageMenuResourceEditor

**Direct DAT dependencies:** DAT.Core.Types, DAT.Runtime.LanguagePack, DAT.Scan.TextCodec

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 563 lines. Changes require the integration contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### **DAT.Integration.Package**

**File:** source\integration\DAT.Integration.Package.pas

**What it is:** Builds the broader integration plan and coordinates package/source actions, generated artifacts, reset behavior, and safety transactions across supported integration modes.

**Why it matters:** This unit exists to keep component-kit generation, controlled Delphi project integration, build, and deployment in the integration layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TIntegrationPackageGenerator

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.RuntimePack, DAT.Core.TranslationWorkspace, DAT.Core.Types, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Scan.CatalogMerge, DAT.Scan.Project, DAT.Scan.Types

**Direct production consumers:** DAT.Studio.MainForm

**Named test consumers:** FoundationSmokeTests

**Change and validation risk:** The file contains 492 lines. Changes require the integration contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 23.10 Studio units

These units own the FMX operator interface, Setup Wizard state machine, Localization Review, and Studio self-translation. The entries are ordered by unit name.

### DAT.Studio.LocalizationReview

**File:** source\studio\DAT.Studio.LocalizationReview.pas

**What it is:** Implements the Localization Review FMX window: findings, glossary terms, translation suggestions, layout proposals, decision persistence, HTML package generation, and controlled return to Wizard processing.

**Why it matters:** This unit exists to keep the FMX operator interface, Setup Wizard state machine, Localization Review, and Studio self-translation in the studio layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TfrmLocalizationReview

**Direct DAT dependencies:** DAT.Core.Glossary, DAT.Core.Types, DAT.Review.ApplicationStrings, DAT.Review.Localization, DAT.Runtime.LanguagePack

**Direct production consumers:** DAT.Studio.SetupWizard

**Named test consumers:** StudioFormSmokeTests

**Change and validation risk:** The file contains 610 lines. Changes require the studio contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

### DAT.Studio.MainForm

**File:** source\studio\DAT.Studio.MainForm.pas

**What it is:** Implements the landing screen and seven-page Maintenance Studio. It coordinates project detection, scanning, catalog editing, provider translation, validation, export, integration, settings, keyboard defaults, and status reporting.

**Why it matters:** This unit exists to keep the FMX operator interface, Setup Wizard state machine, Localization Review, and Studio self-translation in the studio layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TfrmTranslationStudio

**Direct DAT dependencies:** DAT.Core.BuildInfo, DAT.Core.CatalogJson, DAT.Core.LocaleFacts, DAT.Core.ProjectDetection, DAT.Core.RuntimePack, DAT.Core.Terminology,

DAT.Core.TranslationWorkspace, DAT.Core.Types, DAT.Integration.BuildDeploy, DAT.Integration.ComponentPackage, DAT.Integration.Package, DAT.Provider.Client, DAT.Provider.CredentialStore, DAT.Provider.Settings, DAT.Provider.Types, DAT.Runtime.LanguagePack, DAT.Scan.CatalogMerge, DAT.Scan.Project, DAT.Scan.Types, DAT.Studio.SetupWizard, DAT.Studio.Translation, DAT.Validation.Catalog

**Direct production consumers:** No production DAT unit imports it directly; it is an executable/package entry point or optional adapter.

**Named test consumers:** StudioFormSmokeTests

**Change and validation risk:** The file contains 2,660 lines. Changes require the studio contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Studio.SetupWizard

**File:** source\studio\DAT.Studio.SetupWizard.pas

**What it is:** Implements the eight-step Setup Wizard state machine, prerequisite validation, scan/translation/review/final-processing sequence, cancellation boundaries, safety backup, kit generation, deployment, and Finish behavior.

**Why it matters:** This unit exists to keep the FMX operator interface, Setup Wizard state machine, Localization Review, and Studio self-translation in the studio layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** TfrmSetupWizard

**Direct DAT dependencies:** DAT.Core.AtomicFile, DAT.Core.BuildInfo, DAT.Core.CatalogJson, DAT.Core.Glossary, DAT.Core.Hyphenation, DAT.Core.LocaleFacts, DAT.Core.ProjectDetection, DAT.Core.RuntimePack, DAT.Core.SharedDictionary, DAT.Core.Terminology, DAT.Core.TranslationMemory, DAT.Core.TranslationWorkspace, DAT.Core.Types, DAT.Integration.BuildDeploy, DAT.Integration.ComponentPackage, DAT.Provider.Client, DAT.Provider.CredentialStore, DAT.Provider.Settings, DAT.Provider.Types, DAT.Review.Localization, DAT.Runtime.LanguagePack, DAT.Scan.CatalogMerge, DAT.Scan.DomainProfile, DAT.Scan.Project, DAT.Scan.Types, DAT.Studio.LocalizationReview, DAT.Validation.Catalog

**Direct production consumers:** DAT.Studio.MainForm

**Named test consumers:** StudioFormSmokeTests

**Change and validation risk:** The file contains 2,476 lines. Changes require the studio contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## DAT.Studio.Translation

**File:** source\studio\DAT.Studio.Translation.pas

**What it is:** Self-localizes the Studio interface by loading its own packs, applying them to FMX forms, maintaining the interface-language menu, and keeping Studio language state separate from target-project catalogs.

**Why it matters:** This unit exists to keep the FMX operator interface, Setup Wizard state machine, Localization Review, and Studio self-translation in the studio layer instead of coupling that responsibility to unrelated UI or framework code.

**Public surface:** InitializeStudioTranslation, ApplyStudioTranslation, SelectStudioLanguageMenuItem, StudioTranslationRuntime

**Direct DAT dependencies:** DAT.Runtime.FMX, DAT.Runtime.Manager

**Direct production consumers:** DAT.Studio.MainForm

**Named test consumers:** Coverage is indirect through subsystem and integration tests.

**Change and validation risk:** The file contains 103 lines. Changes require the studio contract tests, a clean package/application build, and a runtime regression pass for both VCL and FMX whenever shared behavior changes.

## 24. Test, Spike, and Contract Pascal Program Reference

Every Delphi test DPR is listed below. Test-support PAS/DFM/FMX units and formal contract fixtures follow. These files are not runtime dependencies; they supply isolated evidence and intentionally artificial application behavior.

### 24.1 Test executable programs

#### ApplicationStringSmokeTests

**File:** tools\tests\ApplicationStringSmokeTests.dpr

**What it verifies:** Verifies application-owned and dynamic string contracts that are not ordinary designer properties.

**DAT units exercised directly:** DAT.Core.Types, DAT.Review.ApplicationStrings

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

#### AtomicPersistenceSmokeTests

**File:** tools\tests\AtomicPersistenceSmokeTests.dpr

**What it verifies:** Exercises atomic save, previous-file recovery, corruption quarantine, and interrupted-write behavior.

**DAT units exercised directly:** DAT.Core.AtomicFile

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

#### CalendarTermSmokeTests

**File:** tools\tests\CalendarTermSmokeTests.dpr

**What it verifies:** Checks locale-aware calendar terminology and ambiguous weekday/month handling.

**DAT units exercised directly:** DAT.Provider.CalendarTerms

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## ContextBatchingSmokeTests

**File:** tools\tests\ContextBatchingSmokeTests.dpr

**What it verifies:** Checks that provider batches preserve compatible contexts and obey configured size limits.

**DAT units exercised directly:** DAT.Provider.Batching

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## ContextSmokeTests

**File:** tools\tests\ContextSmokeTests.dpr

**What it verifies:** Checks semantic role, concept, description, and confidence inference for representative UI text.

**DAT units exercised directly:** DAT.Scan.Context, DAT.Scan.DomainProfile, DAT.Scan.Types

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## FMXCaptionRewriteSmokeTests

**File:** tools\tests\FMXCaptionRewriteSmokeTests.dpr

**What it verifies:** Exercises the F M X Caption Rewrite contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.TemplateRewrite, DAT.Runtime.TemplateRewrite.FMX

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## FMXDesignStreamingTests

**File:** tools\tests\FMXDesignStreamingTests.dpr

**What it verifies:** Exercises the F M X Design Streaming contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.FMX, DAT.Components.FMX.LanguageSelector, DAT.Core.AtomicFile, DAT.Core.Diagnostics, DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXDialogTranslationSmokeTests

**File:** tools\tests\FMXDialogTranslationSmokeTests.dpr

**What it verifies:** Exercises the F M X Dialog Translation contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.FMX, DAT.Core.AtomicFile, DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXDiscoverySmokeTests

**File:** tools\tests\FMXDiscoverySmokeTests.dpr

**What it verifies:** Exercises the F M X Discovery contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.FMX, DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXGridOrderSmokeTests

**File:** tools\tests\FMXGridOrderSmokeTests.dpr

**What it verifies:** Exercises the F M X Grid Order contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack



**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXLanguageManagerTests

**File:** tools\tests\FMXLanguageManagerTests.dpr

**What it verifies:** Exercises the F M X Language Manager contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.FMX, DAT.Core.AtomicFile, DAT.LifecycleSpike.Trace, DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXManagerLifecycleSpikeTests

**File:** tools\tests\FMXManagerLifecycleSpikeTests.dpr

**What it verifies:** Exercises the F M X Manager Lifecycle Spike contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.FMX.Spike, DAT.LifecycleSpike.Trace, DAT.Runtime.FMX, DAT.Runtime.LanguagePack

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXMenuOrderSmokeTests

**File:** tools\tests\FMXMenuOrderSmokeTests.dpr

**What it verifies:** Exercises the F M X Menu Order contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXRightToLeftSmokeTests

**File:** tools\tests\FMXRightToLeftSmokeTests.dpr

**What it verifies:** Exercises the F M X Right To Left contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXRuntimeSmokeTests

**File:** tools\tests\FMXRuntimeSmokeTests.dpr

**What it verifies:** Exercises the F M X Runtime contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### FMXSplashHookSmokeTests

**File:** tools\tests\FMXSplashHookSmokeTests.dpr

**What it verifies:** Exercises the F M X Splash Hook contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.Preference, DAT.Runtime.SplashTranslation, DAT.Runtime.SplashTranslation.FMX

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## FMXStyledRuntimeSmokeTests

**File:** tools\tests\FMXStyledRuntimeSmokeTests.dpr

**What it verifies:** Exercises the F M X Styled Runtime contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack, DAT.Runtime.LayoutOverrides

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## FMXWrapSmokeTests

**File:** tools\tests\FMXWrapSmokeTests.dpr

**What it verifies:** Exercises the F M X Wrap contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.FMX, DAT.Runtime.LanguagePack

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## FormScanContracts

**File:** tools\tests\FormScanContracts.dpr

**What it verifies:** Runs the formal DFM/FMX scan contract fixtures and compares the canonical expected results.

**DAT units exercised directly:** DAT.Core.Types, DAT.Scan.Context, DAT.Scan.DomainProfile, DAT.Scan.FormText, DAT.Scan.Quality, DAT.Scan.Rules, DAT.Scan.TextCodec, DAT.Scan.Types

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## FoundationSmokeTests

**File:** tools\tests\FoundationSmokeTests.dpr

**What it verifies:** Covers foundational types, JSON, locale, workspace, validation, and runtime-pack construction contracts.

**DAT units exercised directly:** DAT.Core.AITranslation, DAT.Core.AtomicFile, DAT.Core.CatalogJson, DAT.Core.Glossary, DAT.Core.Hyphenation, DAT.Core.ProjectDetection, DAT.Core.RuntimePack, DAT.Core.Terminology, DAT.Core.TranslationWorkspace, DAT.Core.Types, DAT.Integration.BuildDeploy, DAT.Integration.ComponentPackage, DAT.Integration.DelphiSource, DAT.Integration.MenuResource, DAT.Integration.Package, DAT.Provider.Batching, DAT.Provider.CalendarTerms, DAT.Provider.Client, DAT.Provider.LanguageCodes, DAT.Provider.Placeholders, DAT.Provider.Retry, DAT.Provider.Types, DAT.Review.ApplicationStrings, DAT.Review.CodeGeometry, DAT.Review.Localization, DAT.Review.TextMeasurement, DAT.Review.TextMeasurement.FMX, DAT.Review.TextMeasurement.GDI, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference, DAT.Scan.CatalogMerge, DAT.Scan.Context, DAT.Scan.DomainProfile, DAT.Scan.FormText, DAT.Scan.PascalResources, DAT.Scan.Project, DAT.Scan.Quality, DAT.Scan.Rules, DAT.Scan.TextCodec, DAT.Scan.Types, DAT.Validation.Catalog

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### HyphenationSmokeTests

**File:** tools\tests\HyphenationSmokeTests.dpr

**What it verifies:** Exercises the Hyphenation contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.Hyphenation, DAT.Core.RuntimePack, DAT.Core.Types

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### LanguageCodeSmokeTests

**File:** tools\tests\LanguageCodeSmokeTests.dpr

**What it verifies:** Exercises the Language Code contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Provider.LanguageCodes

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## LanguageManagerCoreTests

**File:** tools\tests\LanguageManagerCoreTests.dpr

**What it verifies:** Exercises the Language Manager Core contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Core.AtomicFile, DAT.Core.Diagnostics, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## LayoutContracts

**File:** tools\tests\LayoutContracts.dpr

**What it verifies:** Exercises the Layout Contracts contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.CatalogJson, DAT.Core.Types, DAT.Review.CodeGeometry, DAT.Review.Localization, DAT.Review.TextMeasurement, DAT.Review.TextMeasurement.FMX, DAT.Review.TextMeasurement.GDI, DAT.Scan.TextCodec

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## LayoutFittingSmokeTests

**File:** tools\tests\LayoutFittingSmokeTests.dpr

**What it verifies:** Exercises the Layout Fitting contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.CatalogJson, DAT.Core.Types, DAT.Review.CodeGeometry, DAT.Review.Localization, DAT.Review.TextMeasurement, DAT.Review.TextMeasurement.FMX, DAT.Review.TextMeasurement.GDI, DAT.Scan.TextCodec

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## LayoutOverrideSmokeTests

**File:** tools\tests\LayoutOverrideSmokeTests.dpr

**What it verifies:** Exercises the Layout Override contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.LayoutOverrides, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## PackLayoutSmokeTests

**File:** tools\tests\PackLayoutSmokeTests.dpr

**What it verifies:** Exercises the Pack Layout contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.Hyphenation, DAT.Core.RuntimePack, DAT.Core.Types

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## PlaceholderSmokeTests

**File:** tools\tests\PlaceholderSmokeTests.dpr

**What it verifies:** Exercises the Placeholder contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Provider.Placeholders

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## ProposalDecisionSmokeTests

**File:** tools\tests\ProposalDecisionSmokeTests.dpr

**What it verifies:** Exercises the Proposal Decision contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.CatalogJson, DAT.Core.Types, DAT.Review.CodeGeometry, DAT.Review.Localization, DAT.Review.TextMeasurement, DAT.Review.TextMeasurement.FMX, DAT.Review.TextMeasurement.GDI, DAT.Scan.TextCodec

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### ProviderGlossarySmokeTests

**File:** tools\tests\ProviderGlossarySmokeTests.dpr

**What it verifies:** Exercises the Provider Glossary contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.Glossary, DAT.Core.Types, DAT.Provider.Glossary

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### RetrySmokeTests

**File:** tools\tests\RetrySmokeTests.dpr

**What it verifies:** Exercises the Retry contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Provider.Retry

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### ScannerSmokeTests

**File:** tools\tests\ScannerSmokeTests.dpr

**What it verifies:** Exercises the Scanner contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.Types, DAT.Scan.Context, DAT.Scan.DomainProfile, DAT.Scan.PascalResources, DAT.Scan.Quality, DAT.Scan.Rules, DAT.Scan.TextCodec, DAT.Scan.Types

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## StudioFormSmokeTests

**File:** tools\tests\StudioFormSmokeTests.dpr

**What it verifies:** Streams and creates the production Studio forms to detect missing components, invalid properties, and form-resource regressions.

**DAT units exercised directly:** DAT.Studio.LocalizationReview, DAT.Studio.MainForm, DAT.Studio.SetupWizard

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## TemplateRewriteSmokeTests

**File:** tools\tests\TemplateRewriteSmokeTests.dpr

**What it verifies:** Exercises the Template Rewrite contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.TemplateRewrite

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## TranslationMemorySmokeTests

**File:** tools\tests\TranslationMemorySmokeTests.dpr

**What it verifies:** Exercises the Translation Memory contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Core.Glossary, DAT.Core.Interchange, DAT.Core.TranslationMemory, DAT.Core.Types

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## VCLCaptionInterceptSmokeTests

**File:** tools\tests\VCLCaptionInterceptSmokeTests.dpr

**What it verifies:** Exercises the V C L Caption Intercept contract as an isolated Delphi console or GUI regression program.



**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.TemplateRewrite, DAT.Runtime.TemplateRewrite.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLDesignStreamingTests

**File:** tools\tests\VCLDesignStreamingTests.dpr

**What it verifies:** Exercises the V C L Design Streaming contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.VCL, DAT.Components.VCL.LanguageSelector, DAT.Core.AtomicFile, DAT.Core.Diagnostics, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLDiscoverySmokeTests

**File:** tools\tests\VCLDiscoverySmokeTests.dpr

**What it verifies:** Exercises the V C L Discovery contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.VCL, DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLLanguageManagerTests

**File:** tools\tests\VCLLanguageManagerTests.dpr

**What it verifies:** Exercises the V C L Language Manager contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.VCL, DAT.Core.AtomicFile, DAT.LifecycleSpike.Trace, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLManagerLifecycleSpikeTests

**File:** tools\tests\VCLManagerLifecycleSpikeTests.dpr

**What it verifies:** Exercises the V C L Manager Lifecycle Spike contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.VCL.Spike, DAT.LifecycleSpike.Trace, DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLManagerMDILifecycleSpikeTests

**File:** tools\tests\VCLManagerMDILifecycleSpikeTests.dpr

**What it verifies:** Exercises the V C L Manager M D I Lifecycle Spike contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.VCL.Spike, DAT.LifecycleSpike.Trace, DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLRightToLeftSmokeTests

**File:** tools\tests\VCLRightToLeftSmokeTests.dpr

**What it verifies:** Exercises the V C L Right To Left contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLRuntimeSmokeTests

**File:** tools\tests\VCLRuntimeSmokeTests.dpr

**What it verifies:** Exercises the V C L Runtime contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLSplashHookSmokeTests

**File:** tools\tests\VCLSplashHookSmokeTests.dpr

**What it verifies:** Exercises the V C L Splash Hook contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.Preference, DAT.Runtime.SplashTranslation, DAT.Runtime.SplashTranslation.VCL, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

### VCLSplashSmokeTests

**File:** tools\tests\VCLSplashSmokeTests.dpr

**What it verifies:** Exercises the V C L Splash contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Components.Core, DAT.Components.VCL, DAT.Runtime.LanguagePack, DAT.Runtime.Manager, DAT.Runtime.Preference, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## VCLStatusPanelInterceptTests

**File:** tools\tests\VCLStatusPanelInterceptTests.dpr

**What it verifies:** Exercises the V C L Status Panel Intercept contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## VCLWrapSmokeTests

**File:** tools\tests\VCLWrapSmokeTests.dpr

**What it verifies:** Exercises the V C L Wrap contract as an isolated Delphi console or GUI regression program.

**DAT units exercised directly:** DAT.Runtime.LanguagePack, DAT.Runtime.VCL

**Why it is important:** This executable isolates one contract so a regression produces a focused failure rather than being hidden inside a full application run.

**Required use:** Compile and run it through the maintained verification scripts or documented build matrix whenever its named subsystem changes.

## 24.2 Test-support Pascal units

### FMXDesignHost.pas

**File:** tools\tests\design\FMXDesignHost.pas

**Program or unit:** FMXDesignHost

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** DAT.Components.FMX, DAT.Components.FMX.LanguageSelector

### VCLDesignHost.pas

**File:** tools\tests\design\VCLDesignHost.pas

**Program or unit:** VCLDesignHost

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** DAT.Components.VCL, DAT.Components.VCL.LanguageSelector

**DAT.LifecycleSpike.Trace.pas**

**File:** tools\tests\lifecycle\DAT.LifecycleSpike.Trace.pas

**Program or unit:** DAT.LifecycleSpike.Trace

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** None; this fixture supplies target behavior or source text for another test.

**FMXLifecycle.Form.pas**

**File:** tools\tests\lifecycle\FMXLifecycle.Form.pas

**Program or unit:** FMXLifecycle.Form

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** DAT.LifecycleSpike.Trace

**FMXLifecycle.InheritedForm.pas**

**File:** tools\tests\lifecycle\FMXLifecycle.InheritedForm.pas

**Program or unit:** FMXLifecycle.InheritedForm

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** None; this fixture supplies target behavior or source text for another test.

**VCLLifecycle.Form.pas**

**File:** tools\tests\lifecycle\VCLLifecycle.Form.pas

**Program or unit:** VCLLifecycle.Form

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** DAT.LifecycleSpike.Trace

**VCLLifecycle.InheritedForm.pas**

**File:** tools\tests\lifecycle\VCLLifecycle.InheritedForm.pas

**Program or unit:** VCLLifecycle.InheritedForm

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** None; this fixture supplies target behavior or source text for another test.

**VCLLifecycle.MDChild.pas**

**File:** tools\tests\lifecycle\VCLLifecycle.MDChild.pas

**Program or unit:** VCLLifecycle.MDChild

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** DAT.LifecycleSpike.Trace

**VCLLifecycle.MDIMain.pas**

**File:** tools\tests\lifecycle\VCLLifecycle.MDIMain.pas

**Program or unit:** VCLLifecycle.MDIMain

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** DAT.LifecycleSpike.Trace

**24.3 Formal scan/layout Pascal fixtures****35\_code\_positioned\_control\_is\_left\_alone.pas**

**File:** contracts\layout\35\_code\_positioned\_control\_is\_left\_alone.pas

**Program or unit:** code\_positioned\_control\_is\_left\_alone\_fm

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** None; this fixture supplies target behavior or source text for another test.

**vcl\_34\_code\_positioned\_control\_is\_left\_alone.pas**

**File:** contracts\layout\vcl\_34\_code\_positioned\_control\_is\_left\_alone.pas

**Program or unit:** vcl\_34\_code\_positioned\_control\_is\_left\_alone

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** None; this fixture supplies target behavior or source text for another test.

**scan\_fixture.pas**

**File:** contracts\pascalscan\scan\_fixture.pas

**Program or unit:** ScanFixture

**Purpose:** Support source or fixture used by lifecycle, design-streaming, scan, or layout contract tests; it is not shipped as an application runtime dependency.

**DAT dependencies:** None; this fixture supplies target behavior or source text for another test.

## 25. Delphi Project and Package File Reference

### 25.1 Application, command-line, and package entry points

#### **DelphiAppTranslationStudio.dpr**

**File:** DelphiAppTranslationStudio.dpr

**What it is:** FMX application entry point that initializes the Studio translation runtime and creates the main Studio form.

**Why it matters:** Defines process startup ordering and the first form/language initialization boundary.

#### **DelphiAppTranslationStudio.dproj**

**File:** DelphiAppTranslationStudio.dproj

**What it is:** RAD Studio project metadata, unit search paths, platforms/configurations, resources, outputs, and main form identity.

**Why it matters:** A build can compile different source or write to a different output when DPROJ configuration drifts, even if DPR is unchanged.

#### **DATBatch.dpr**

**File:** tools\DATBatch.dpr

**What it is:** Command-line/batch entry point for supported noninteractive translation operations.

**Why it matters:** Provides automation without coupling scripts directly to Studio form classes.

#### **DATLanguageManagerFMXDesign.dpk**

**File:** packages\design\DATLanguageManagerFMXDesign.dpk

**Package kind:** Design-time

**Requires:** rtl, fmx, designide, DATLanguageManagerCoreRuntime, DATLanguageManagerFMXRuntime

**Contains:** DAT.Design.FMX.Register in '..\..\source\design\DAT.Design.FMX.Register.pas

**Why it exists:** Separates framework-neutral, framework-runtime, and IDE-only registration code so deployed applications never depend on design-time units and RAD Studio can stream the published components correctly.

**Companion DPROJ:** packages\design\DATLanguageManagerFMXDesign.dproj - platform/configuration/output metadata for this package.

#### **DATLanguageManagerVCLDesign.dpk**

**File:** packages\design\DATLanguageManagerVCLDesign.dpk

**Package kind:** Design-time



**Requires:** rtl, vcl, designide, DATLanguageManagerCoreRuntime, DATLanguageManagerVCLRuntime

**Contains:** DAT.Design.VCL.Register in '..\..\source\design\DAT.Design.VCL.Register.pas

**Why it exists:** Separates framework-neutral, framework-runtime, and IDE-only registration code so deployed applications never depend on design-time units and RAD Studio can stream the published components correctly.

**Companion DPROJ:** packages\design\DATLanguageManagerVCLDesign.dproj - platform/configuration/output metadata for this package.

### DATLanguageManagerCoreRuntime.dpk

**File:** packages\runtime\DATLanguageManagerCoreRuntime.dpk

**Package kind:** Runtime

**Requires:** rtl

**Contains:** DAT.Core.AtomicFile in '..\..\source\core\DAT.Core.AtomicFile.pas, DAT.Core.Diagnostics in '..\..\source\core\DAT.Core.Diagnostics.pas, DAT.Runtime.LanguagePack in '..\..\source\runtime\DAT.Runtime.LanguagePack.pas, DAT.Runtime.Preference in '..\..\source\runtime\DAT.Runtime.Preference.pas, DAT.Runtime.LayoutOverrides in '..\..\source\runtime\DAT.Runtime.LayoutOverrides.pas, DAT.Runtime.Manager in '..\..\source\runtime\DAT.Runtime.Manager.pas, DAT.Runtime.SplashTranslation in '..\..\source\runtime\DAT.Runtime.SplashTranslation.pas, DAT.Runtime.TemplateRewrite in '..\..\source\runtime\DAT.Runtime.TemplateRewrite.pas, DAT.Components.Core in '..\..\source\components\DAT.Components.Core.pas

**Why it exists:** Separates framework-neutral, framework-runtime, and IDE-only registration code so deployed applications never depend on design-time units and RAD Studio can stream the published components correctly.

**Companion DPROJ:** packages\runtime\DATLanguageManagerCoreRuntime.dproj - platform/configuration/output metadata for this package.

### DATLanguageManagerFMXRuntime.dpk

**File:** packages\runtime\DATLanguageManagerFMXRuntime.dpk

**Package kind:** Runtime

**Requires:** rtl, fmx, DATLanguageManagerCoreRuntime

**Contains:** DAT.Runtime.FMX in '..\..\source\runtime\DAT.Runtime.FMX.pas, DAT.Runtime.SplashTranslation.FMX in '..\..\source\runtime\DAT.Runtime.SplashTranslation.FMX.pas, DAT.Runtime.TemplateRewrite.FMX in '..\..\source\runtime\DAT.Runtime.TemplateRewrite.FMX.pas, DAT.Components.FMX in '..\..\source\components\DAT.Components.FMX.pas

\source\components\DAT.Components.FMX.pas, DAT.Components.FMX.LanguageSelector in '..\..\source\components\DAT.Components.FMX.LanguageSelector.pas

**Why it exists:** Separates framework-neutral, framework-runtime, and IDE-only registration code so deployed applications never depend on design-time units and RAD Studio can stream the published components correctly.

**Companion DPROJ:** packages\runtime\DATLanguageManagerFMXRuntime.dproj - platform/configuration/output metadata for this package.

### DATLanguageManagerVCLRuntime.dpk

**File:** packages\runtime\DATLanguageManagerVCLRuntime.dpk

**Package kind:** Runtime

**Requires:** rtl, vcl, DATLanguageManagerCoreRuntime

**Contains:** DAT.Runtime.VCL in '..\..\source\runtime\DAT.Runtime.VCL.pas, DAT.Runtime.SplashTranslation.VCL in '..\..\source\runtime\DAT.Runtime.SplashTranslation.VCL.pas, DAT.Runtime.TemplateRewrite.VCL in '..\..\source\runtime\DAT.Runtime.TemplateRewrite.VCL.pas, DAT.Components.VCL in '..\..\source\components\DAT.Components.VCL.pas, DAT.Components.VCL.LanguageSelector in '..\..\source\components\DAT.Components.VCL.LanguageSelector.pas

**Why it exists:** Separates framework-neutral, framework-runtime, and IDE-only registration code so deployed applications never depend on design-time units and RAD Studio can stream the published components correctly.

**Companion DPROJ:** packages\runtime\DATLanguageManagerVCLRuntime.dproj - platform/configuration/output metadata for this package.

## 26. JSON Schema and Form Resource Reference

### 26.1 JSON schemas

#### development-project.schema.json

**File:** source\schemas\development-project.schema.json

**What it defines:** Delphi App Translation Studio Project

**Top-level properties:** applicationId, applicationVersion, entries, framework, locale, schemaVersion, sourceLanguage

**Required top-level properties:** schemaVersion, applicationId, framework, sourceLanguage, entries

**Change rule:** A schema change requires compatible reader/writer changes, version handling, fixtures, round-trip tests, and release notes; never change emitted structure alone.

**layout-proposal.schema.json**

**File:** source\schemas\layout-proposal.schema.json

**What it defines:** Delphi App Translation runtime layout proposal

**Top-level properties:** acceptedSafeCount, advisoryOnly, applicationId, languageCode, proposals, runtimeApplication, schemaVersion, targetSourceModified

**Required top-level properties:** schemaVersion, applicationId, languageCode, advisoryOnly, targetSourceModified, proposals

**Change rule:** A schema change requires compatible reader/writer changes, version handling, fixtures, round-trip tests, and release notes; never change emitted structure alone.

**multilingual-layout-envelope.schema.json**

**File:** source\schemas\multilingual-layout-envelope.schema.json

**What it defines:** Delphi App Translation multilingual layout envelope

**Top-level properties:** advisoryOnly, controls, languages, purpose, schemaVersion

**Required top-level properties:** schemaVersion, purpose, languages, advisoryOnly, controls

**Change rule:** A schema change requires compatible reader/writer changes, version handling, fixtures, round-trip tests, and release notes; never change emitted structure alone.

**project-glossary.schema.json**

**File:** source\schemas\project-glossary.schema.json

**What it defines:** Delphi App Translation project glossary

**Top-level properties:** applicationId, schemaVersion, sourceLanguage, targetLanguage, terms

**Required top-level properties:** schemaVersion, applicationId, sourceLanguage, targetLanguage, terms

**Change rule:** A schema change requires compatible reader/writer changes, version handling, fixtures, round-trip tests, and release notes; never change emitted structure alone.

**runtime-language-pack.schema.json**

**File:** source\schemas\runtime-language-pack.schema.json

**What it defines:** Delphi App Translation Runtime Language Pack

**Top-level properties:** applicationId, applicationVersion, fontColors, framework, language, layout, locale, schemaVersion, sourceCatalogChecksum, sourceLanguage, sourceStrings, sourceTemplates, sources, strings, templates

**Required top-level properties:** schemaVersion, applicationId, framework, sourceLanguage, sourceCatalogChecksum, language, strings

**Change rule:** A schema change requires compatible reader/writer changes, version handling, fixtures, round-trip tests, and release notes; never change emitted structure alone.

## 26.2 Production FMX form resources

### DAT.Studio.MainForm.fmx

**File:** source\studio\DAT.Studio.MainForm.fmx

**What it is:** Designer-authored landing screen and seven Maintenance pages, navigation rail, dialogs, status, and control defaults.

**Why it matters:** It is the editable IDE layout authority paired with the same-base-name PAS class. Runtime code must not replace this designer ownership without explicit approval.

### DAT.Studio.SetupWizard.fmx

**File:** source\studio\DAT.Studio.SetupWizard.fmx

**What it is:** Designer-authored eight-step Wizard, step rail, page controls, keyboard defaults, authorization, and final-processing layout.

**Why it matters:** It is the editable IDE layout authority paired with the same-base-name PAS class. Runtime code must not replace this designer ownership without explicit approval.

### DAT.Studio.LocalizationReview.fmx

**File:** source\studio\DAT.Studio.LocalizationReview.fmx

**What it is:** Designer-authored findings/glossary/suggestions/layout-decision review window.

**Why it matters:** It is the editable IDE layout authority paired with the same-base-name PAS class. Runtime code must not replace this designer ownership without explicit approval.

## 26.3 Test form resources

### VCLDesignHost.dfm

**File:** tools\tests\design\VCLDesignHost.dfm

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

### VCLLifecycle.Form.dfm

**File:** tools\tests\lifecycle\VCLLifecycle.Form.dfm

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

### **VCLLifecycle.InheritedForm.dfm**

**File:** tools\tests\lifecycle\VCLLifecycle.InheritedForm.dfm

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

### **VCLLifecycle.MDIChild.dfm**

**File:** tools\tests\lifecycle\VCLLifecycle.MDIChild.dfm

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

### **VCLLifecycle.MDIMain.dfm**

**File:** tools\tests\lifecycle\VCLLifecycle.MDIMain.dfm

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

### **FMXDesignHost.fmx**

**File:** tools\tests\design\FMXDesignHost.fmx

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

### **FMXLifecycle.Form.fmx**

**File:** tools\tests\lifecycle\FMXLifecycle.Form.fmx

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

### **FMXLifecycle.InheritedForm.fmx**

**File:** tools\tests\lifecycle\FMXLifecycle.InheritedForm.fmx

**What it is:** Designer resource streamed by a lifecycle or design test.

**Why it matters:** Proves published properties, inheritance, object names, lifecycle behavior, and form-resource compatibility.

## 27. Build, Verification, and Documentation Tool Reference

### 27.1 Build, verification, distribution, and guide tools

#### **build\_packages.ps1**

**File:** tools\build\_packages.ps1

**What it is:** Builds the five runtime/design package projects in controlled platform/configuration order.

**Why it matters:** Package installation and ComponentSource correctness depend on reproducible package outputs.

#### **verify\_all.ps1**

**File:** tools\verify\_all.ps1

**What it is:** Runs the broad engineering verification suite.

**Why it matters:** Primary pre-release gate across foundation, scanner, runtime, components, Studio, and artifacts.

#### **check\_shipped\_units\_complete.ps1**

**File:** tools\check\_shipped\_units\_complete.ps1

**What it is:** Compares required runtime/component units with shipped packages/kits/distributions.

**Why it matters:** Prevents a compile/runtime failure caused by an omitted dependency.

#### **check\_build\_paths\_agree.ps1**

**File:** tools\check\_build\_paths\_agree.ps1

**What it is:** Checks project, script, and documentation output/search-path agreement.

**Why it matters:** Prevents testing one binary while shipping another.

#### **check\_source\_encoding.ps1**

**File:** tools\check\_source\_encoding.ps1

**What it is:** Audits source encoding and known text hazards.

**Why it matters:** Scanner/provider/UI correctness depends on stable Unicode source.

**build\_source\_distribution.ps1**

**File:** tools\build\_source\_distribution.ps1

**What it is:** Creates the source-only distribution with approved files and structure.

**Why it matters:** Distribution must be complete while excluding secrets, transient workspaces, and unintended binaries.

**run\_form\_scan\_contracts.ps1**

**File:** tools\run\_form\_scan\_contracts.ps1

**What it is:** Builds/runs formal DFM/FMX scan contracts.

**Why it matters:** Protects canonical scan keys, counts, inclusion/exclusion, and context behavior.

**run\_layout\_contracts.ps1**

**File:** tools\run\_layout\_contracts.ps1

**What it is:** Builds/runs layout and direction contracts.

**Why it matters:** Protects designer ownership, wrapping, fitting, RTL, and code-positioned exclusions.

**run\_pascal\_scan\_contracts.ps1**

**File:** tools\run\_pascal\_scan\_contracts.ps1

**What it is:** Builds/runs Pascal resourcestring/runtime-assignment contracts.

**Why it matters:** Protects text extraction without executing target source or translating arbitrary literals.

**Install-DATLanguageManagerComponents.ps1**

**File:** tools\Install-DATLanguageManagerComponents.ps1

**What it is:** Builds/locates and guides installation of design components.

**Why it matters:** Keeps Tool Palette setup repeatable without copying arbitrary BPLs into RAD Studio folders.

**render\_guides\_pdf.py**

**File:** tools\render\_guides\_pdf.py

**What it is:** Converts documentation DOCX files to companion PDFs through LibreOffice.

**Why it matters:** Keeps printable output tied to editable source documents.

**build\_user\_guide.py**

**File:** tools\build\_user\_guide.py

**What it is:** Builds the detailed source-driven User Guide and its static printable TOC.

**Why it matters:** Documents actual application behavior and provides deterministic regeneration.

### **build\_wizard\_engineering\_guides.py**

**File:** tools\build\_wizard\_engineering\_guides.py

**What it is:** Builds this Wizard Guide and Engineering Guide from current source inventories and curated architecture descriptions.

**Why it matters:** Keeps exhaustive file/dependency reference synchronized with repository content.

### **finalize\_guides.ps1**

**File:** tools\finalize\_guides.ps1

**What it is:** Performs the maintained guide finalization workflow.

**Why it matters:** Ensures DOCX/PDF artifacts and verification steps remain repeatable.

## **27.2 Important engineering reference documents**

### **Total Stabilization Contracts.md**

**File:** docs\guides\Total Stabilization Contracts.md

**What it is:** Canonical cross-cutting runtime, layout, language-switch, and universal-fix contracts.

**Why it matters:** Read before changing the named subsystem; these records explain contracts that a source signature alone cannot express.

### **Translation Studio Completed Test Report and Remediation Plan.md**

**File:** docs\guides\Translation Studio Completed Test Report and Remediation Plan.md

**What it is:** Evidence, resolved defects, residual risks, and verification results.

**Why it matters:** Read before changing the named subsystem; these records explain contracts that a source signature alone cannot express.

### **TDATLanguageManager Deep-Dive and Implementation Plan.md**

**File:** docs\guides\TDATLanguageManager Deep-Dive and Implementation Plan.md

**What it is:** Manager architecture and staged implementation rationale.

**Why it matters:** Read before changing the named subsystem; these records explain contracts that a source signature alone cannot express.

### **STAGES-1-3-REVIEW-AND-SCANNING-ENGINEERING-NOTES.md**

**File:** docs\STAGES-1-3-REVIEW-AND-SCANNING-ENGINEERING-NOTES.md

**What it is:** Detailed scan/context/review engineering notes.



**Why it matters:** Read before changing the named subsystem; these records explain contracts that a source signature alone cannot express.

### STAGES-4-10-RUNTIME-LAYOUT-ENGINEERING-NOTES.md

**File:** docs\STAGES-4-10-RUNTIME-LAYOUT-ENGINEERING-NOTES.md

**What it is:** Detailed runtime/layout/direction/stabilization engineering notes.

**Why it matters:** Read before changing the named subsystem; these records explain contracts that a source signature alone cannot express.

### Release Checklist.md

**File:** docs\guides\Release Checklist.md

**What it is:** Human release verification checklist complementing automated tests.

**Why it matters:** Read before changing the named subsystem; these records explain contracts that a source signature alone cannot express.

## 28. Dependency Graph Reading Guide

Every production unit entry in Chapter 23 includes direct DAT dependencies and direct production consumers computed from current source. A dependency means the unit imports another DAT unit; a consumer means another production unit imports it. The list is direct, not the complete transitive closure.

Leaf/foundation units may have no DAT dependencies but many consumers. Adapter/entry units may have many dependencies and no production consumer because a DPR/DPK or application code selects them. Test consumers show named DPRs that import the unit directly; indirect coverage is broader.

| Graph signal                           | Likely meaning                       | Required engineering response                                                  |
|----------------------------------------|--------------------------------------|--------------------------------------------------------------------------------|
| Many consumers                         | Shared API or foundational contract. | Treat signature/behavior changes as cross-subsystem and run full verification. |
| Framework-neutral unit imports VCL/FMX | Layer violation.                     | Move framework behavior into the applicator/component adapter.                 |

| Graph signal                                                  | Likely meaning                             | Required engineering response                                                         |
|---------------------------------------------------------------|--------------------------------------------|---------------------------------------------------------------------------------------|
| <b>Runtime imports provider/studio</b>                        | Offline trust-boundary violation.          | Remove network/UI dependency from shipped runtime.                                    |
| <b>Design unit in runtime package</b>                         | IDE-only code can leak into deployment.    | Correct DPK containment/requirements.                                                 |
| <b>ComponentSource missing a direct/transitive dependency</b> | Generated kit cannot compile consistently. | Fix closure computation and shipped-unit check; do not hand-copy one file.            |
| <b>No named tests for high-risk unit</b>                      | Coverage may be only indirect.             | Add focused contract test when behavior cannot be proven by existing subsystem tests. |

## 29. Safe Change Procedure

1. Define the contract and affected ownership layer before editing. Read the relevant reference document and unit entries in Chapter 23.
2. Check Git status and remote currency; preserve unrelated work. Make a pre-change backup for material changes.
3. Trace direct dependencies, consumers, package membership, ComponentSource closure, schemas, and tests. Do not edit only the first file named by a compiler error.
4. Make the smallest universal change. Do not add named-project/version exceptions, runtime UI construction, or helper layers without explicit approval.
5. Update schema/version or compatibility handling when serialized contracts change. Preserve atomic persistence and older valid recovery files.
6. Compile affected foundation/runtime/package/Studio targets and run focused tests first, then the complete verification suite.
7. Run real VCL and FMX pilot applications across LTR/RTL, language-to-language, switch-back, restart, dynamic/HTML, layout, and supported platforms.
8. Update User, Wizard, Engineering, help, release, and test documentation as applicable. Render and visually inspect DOCX/PDF outputs.
9. Review Git diff, stage only intended files, commit clearly, and push configured public/private remotes without rewriting history.

## 30. Engineering Release Checklist

- [ ] Product invariants and trust boundaries remain true.
- [ ] All production PAS units compile in their intended packages/projects.
- [ ] Five package projects build; Win32 Release design BPLs stream components in RAD Studio.
- [ ] Complete ComponentSource closure contains matching shared and framework units.
- [ ] Development catalogs, settings, preferences, glossaries, layout overrides, and packs pass atomic/recovery tests.
- [ ] Canonical scanner totals agree between Wizard and Maintenance for identical inputs.
- [ ] Provider timeout, cancellation, retries, placeholders, language codes, quota/auth errors, and credential isolation pass.
- [ ] Localization Review findings, glossary, suggestions, layout decisions, HTML package, and resume continuation pass.
- [ ] Validation blocks structural errors without pretending to approve language quality.
- [ ] Pack admission rejects wrong application/framework/version/checksum/source contract.
- [ ] VCL and FMX preserve state and designer baseline through every language transition.
- [ ] LTR, RTL, protected LTR tokens, space wrapping, intelligent hyphenation, readable font fitting, and neighbor alignment pass.
- [ ] English/source switch-back repaints current forms/HTML immediately and previous-language content is absent.
- [ ] Startup preference, missing/stale pack fallback, splash, templates, later forms, dialogs, menus, grids, reports, and dynamic text pass.
- [ ] Build/deployment output and actual running executable folder contain the identical intended pack set.
- [ ] Source distribution excludes secrets/transient work and includes required source, packages, schemas, tools, guides, and licenses.
- [ ] Documentation is source-current, rendered, visually inspected, accessible, committed, and pushed.

## 31. Quick Path and Identifier Reference

| Purpose                      | Path / identifier                                                                                 |
|------------------------------|---------------------------------------------------------------------------------------------------|
| Main project                 | DelphiAppTranslationStudio.dproj / DelphiAppTranslationStudio.dpr                                 |
| RAD environment              | C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat                                     |
| Win32 compiler               | C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\dcc32.exe                                      |
| Production source            | source\core, scan, provider, review, validation, runtime, components, design, integration, studio |
| Schemas                      | source\schemas                                                                                    |
| Packages                     | packages\runtime and packages\design                                                              |
| Tests                        | tools\tests                                                                                       |
| Release verification         | tools\verify_all.ps1                                                                              |
| Studio settings              | %LOCALAPPDATA%\DelphiAppTranslationStudio                                                         |
| Project workspace            | %LOCALAPPDATA%\DelphiAppTranslationStudio\Workspaces\<ApplicationId>                              |
| Generated kit                | export\component-integration\<ApplicationId>                                                      |
| Review package               | export\localization-review\<ApplicationId>\<locale>                                               |
| Vendored target dependencies | <Target>\dependencies\DelphiAppTranslation\source                                                 |
| Target packs                 | <EXE>\Localization\Languages                                                                      |
| DeepL credential target      | DelphiAppTranslationStudio/Providers/DeepL                                                        |
| Google credential target     | DelphiAppTranslationStudio/Providers/Google Cloud Translation                                     |